

Kelle: Co-design KV Caching and eDRAM for Efficient LLM Serving in Edge Computing

Tianhua Xia
Tandon School of Engineering
New York University
New York, NY, USA
tx856@nyu.edu

Sai Qian Zhang
Tandon School of Engineering
New York University
New York, NY, USA
sai.zhang@nyu.edu

Abstract

Running Large Language Models (LLMs) on edge devices is crucial for reducing latency, improving real-time processing, and enhancing privacy. By performing inference directly on the device, data does not need to be sent to the cloud, ensuring faster responses and reducing reliance on network connectivity. However, implementing LLMs on edge devices presents challenges, particularly with managing key-value (KV) caches, which plays a pivotal role in LLM serving. As the input text lengthens, the size of the KV cache increases linearly with the sequence length, leading to a significant memory footprint and data access costs. On the other hand, edge devices have limited memory and computational power, making it hard to store and efficiently access the large caches needed for LLM inference.

To mitigate the substantial overhead caused by KV cache, we propose using embedded DRAM (eDRAM) as the primary storage for LLM serving in edge device, which offers higher storage density compared to SRAM. However, to ensure data integrity, eDRAM needs periodic refresh operations, which are power-intensive. To reduce eDRAM costs and improve overall system performance, we propose *Kelle*, a software-hardware co-design solution optimized for deploying LLMs on eDRAM-based edge systems. Combined with our fine-grained memory eviction, recomputation, and refresh control algorithms, the *Kelle* accelerator delivers a 3.9× speedup and 4.5× energy savings compared to existing baseline solutions.

CCS Concepts

• Computer systems organization → Neural networks.

Keywords

Large Language Model, Embedded DRAM

ACM Reference Format:

Tianhua Xia and Sai Qian Zhang. 2025. Kelle: Co-design KV Caching and eDRAM for Efficient LLM Serving in Edge Computing. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*, October 18–22, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3725843.3756071>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '25, October 18–22, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1573-0/2025/10
<https://doi.org/10.1145/3725843.3756071>

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of domains. While cloud-based deployments offer advantages like increased processing power, they also come with limitations, including high communication latency and security risks. As LLMs continue to evolve, it is increasingly important to bring their capabilities directly to edge devices [69]. The integration of LLMs into edge devices not only broadens their accessibility but also ensures robust, customized experiences tailored to individual and industrial needs. This trend is gaining traction not only in academia [12, 25, 90, 91, 94], but also in the industry, where leading companies like Intel [67], NVIDIA [2], Microsoft [77], and Qualcomm [70] are actively exploring similar solutions.

However, implementing LLMs on edge devices presents challenges, particularly with managing key-value (KV) caches [65], which play a critical role to enhance LLM token generation speed. This mechanism involves storing previously computed *Key* and *Value* vectors (KV vectors) during attention calculations and reusing them for generating subsequent tokens. By doing so, it avoids recalculating vectors for earlier tokens with each new token generation. On the other hand, the KV caching incurs a significant memory footprint that grows rapidly as both the model size and the length of generated text increase [32, 99]. For example, when LLaMA 2-7B processes a sequence with the length of 8192 in FP16, the KV cache consumes 4GB of memory, causing the total execution latency to be primarily limited by frequent memory access between on-chip SRAM and off-chip DRAM [59, 90]. This becomes particularly problematic in resource-constrained systems with limited on-chip SRAM capacity, such as edge devices [99]. For example, the Jetson Orin NX edge GPU has only 4MB L3 cache [2].

A simple way to address this issue is by increasing the on-chip SRAM size, which effectively reduces costly off-chip memory access and enhances overall system performance [15, 76]. However, edge devices have limited area and power budgets, and expanding SRAM reduces the resources available for other critical components, such as computational cores [14, 39, 79]. Alternatively, this study explores the use of embedded DRAM (eDRAM) as the primary on-chip storage medium for KV vectors during LLM execution. With fewer transistors per memory cell, such as 3T for eDRAM cells compared to 6T for SRAM cells, eDRAM provides a higher data storage density, resulting in more than twice the capacity [15, 28]. This increased storage density enables a greater on-chip storage capacity within the same chip area. Moreover, eDRAM also offers much lower leakage power than SRAM (around 3.5× according to prior work [17]). These benefits make eDRAM an attractive option for storing KV vectors in edge devices.

Kelle: 共同设计 KV 缓存和 eDRAM, 以实现边缘计算中的高效 LLM 服务

夏天华 纽约大学
Tandon 工程学院 美国
纽约州 tx856@nyu.edu

张赛前 纽约大学 T
andon 工程学院 美国纽
约州纽约市 sai.zhang@n
yu.edu

抽象的

在边缘设备上运行大型语言模型 (LLM) 对于减少延迟、改进实时处理和增强隐私至关重要。通过直接在设备上执行推理, 无需将数据发送到云端, 从而确保更快的响应并减少对网络连接的依赖。然而, 在边缘设备上实现 LLM 会带来挑战, 特别是在管理键值 (KV) 缓存方面, 这在 LLM 服务中发挥着关键作用。随着输入文本的延长, KV 缓存的大小会随着序列长度线性增加, 从而导致显着的内存占用和数据访问成本。另一方面, 边缘设备的内存和计算能力有限, 因此很难存储和有效访问 LLM 推理所需的大型缓存。

为了减轻 KV 缓存造成的大量开销, 我们建议使用嵌入式 DRAM (eDRAM) 作为边缘设备中 LLM 服务的主存储, 与 SRAM 相比, 它提供更高的存储密度。然而, 为了确保数据完整性, eDRAM 需要定期刷新操作, 这是非常耗电的。为了降低 eDRAM 成本并提高整体系统性能, 我们提出了 Kelle, 这是一种软件硬件协同设计解决方案, 针对在基于 eDRAM 的边缘系统上部署 LLM 进行了优化。与我们的细粒度内存驱逐、重新计算和刷新控制算法相结合, 与现有基准解决方案相比, Kelle 加速器可实现 3.9 $\times$ 的加速和 4.5 $\times$ 的节能。

CCS 概念

• 计算机系统组织 $\rightarrow$ 神经网络。

关键词

大语言模型、嵌入式 DRAM

ACM参考格式:

夏天华和张赛千。2025. Kelle: 共同设计 KV 缓存和 eDRAM, 以实现边缘计算中的高效 LLM 服务。第 58 届 IEEE/ACM 国际微架构研讨会 (MICRO '25), 2025 年 10 月 18 日至 22 日, 韩国首尔。ACM, 美国纽约州纽约市, 16 页。https://doi.org/10.1145/3725843.3756071

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MICRO '25, October 18–22, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1573-0/2025/10
https://doi.org/10.1145/3725843.3756071

1 简介

大型语言模型 (LLM) 在广泛的领域中展现了卓越的能力。虽然基于云的部署具有增强处理能力等优势, 但它们也存在局限性, 包括高通信延迟和安全风险。随着大语言模型不断发展, 将其能力直接应用于边缘设备变得越来越重要[69]。将法学硕士集成到边缘设备中不仅扩大了其可访问性, 而且还确保了根据个人和行业需求量身定制的强大的定制体验。这种趋势不仅在学术界受到关注[12,25,90,91,94], 而且在业界也越来越受到关注, 英特尔[67]、NVIDIA[2]、微软[77]和高通[70]等领先公司正在积极探索类似的解决方案。

然而, 在边缘设备上实现 LLM 会带来挑战, 特别是在管理键值 (KV) 缓存方面 [65], 这对于提高 LLM 令牌生成速度起着关键作用。该机制涉及在注意力计算期间存储先前计算的键和值向量 (KV 向量) 并重用它们来生成后续标记。通过这样做, 它可以避免在每个新令牌生成时重新计算早期令牌的向量。另一方面, KV 缓存会产生大量内存占用, 并且随着模型大小和生成文本长度的增加而快速增长 [32, 99]。例如, 当 LLaMA 2-7B 处理 FP16 中长度为 8192 的序列时, KV 缓存消耗 4GB 内存, 导致总执行延迟主要受到片上 SRAM 和片外 DRAM 之间频繁内存访问的限制 [59, 90]。在片上 SRAM 容量有限的资源受限系统 (例如边缘设备) 中, 这变得尤其成问题 [99]。例如, Jetson Orin NX 边缘 GPU 仅具有 4MB 三级缓存 [2]。

解决这个问题的一個简单方法是增加片上 SRAM 的大小, 这可以有效减少昂贵的片外存储器访问并提高整体系统性能 [15, 76]。然而, 边缘设备的面积和功率预算有限, 并且扩展 SRAM 会减少其他关键组件 (例如计算核心) 可用的资源 [14,39, 79]。另外, 本研究探讨了在 LLM 执行期间使用嵌入式 DRAM (eDRAM) 作为 KV 向量的主要片上存储介质。由于每个存储单元的晶体管数量较少, 例如 eDRAM 单元的晶体管数为 3T, 而 SRAM 单元的晶体管数为 6T, 因此 eDRAM 提供了更高的数据存储密度, 从而使容量增加了一倍以上 [15, 28]。这种增加的存储密度可以在相同的芯片面积内实现更大的片上存储容量。此外, eDRAM 还提供比 SRAM 低得多的泄漏功率 (根据先前的工作 [17], 约为 3.5 $\times$)。这些优点使 eDRAM 成为在边缘设备中存储 KV 矢量的有吸引力的选择。

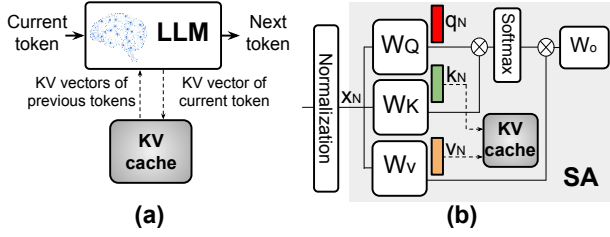


Figure 1: (a) LLM token generation. (b) KV cache for intermediate data storage, where N denotes the token index.

However, a key drawback of eDRAM is the need for periodic refreshes to prevent data loss from leakage. Specifically, refreshing eDRAM cells requires a read-write operation, increasing latency and power consumption, which can significantly impact the efficient deployment of LLMs. To address the challenges of integrating eDRAM, we co-design the KV caching algorithm with the eDRAM-based hardware system, enabling a highly efficient KV cache implementation without compromising accuracy. Our contributions are summarized:

- We propose *Kelle*, an algorithm-system co-design solution for in-device LLM serving on eDRAM-based edge systems. To optimize eDRAM integration cost and improve LLM execution efficiency, we introduce *attention-based eviction and recomputation policy* (AERP) and *two-dimensional adaptive refresh policy* (2DRP) for efficient KV cache implementation (Section 4.1 and 4.2).
- We design a *Kelle accelerator* that utilizes eDRAM as the primary on-chip storage, featuring a customized memory layout. To maximize efficiency, the accelerator integrates a dedicated eDRAM controller and a *systolic evictor* for efficient AERP and 2DRP implementation (Section 5).
- We also introduce the *Kelle scheduler* (Section 6), which adopts an efficient computation pattern to optimize eDRAM data lifetime and LLM serving latency, significantly reducing both eDRAM refresh energy and memory traffic.
- The evaluation results show that *Kelle* achieves a $3.9\times$ speedup and $4.5\times$ energy savings compared to other baseline hardware platforms, while maintaining a negligible impact on LLM accuracy (Section 7, 8).

2 Background and Related Work

2.1 LLM Workflow

Modern LLMs (e.g., Llama series [74, 75], GPT series [11, 65]) are constructed as a stack of transformer decoders, with each decoder comprising two fundamental components: a Self-Attention (SA) block and a feed forward network (FFN). During the LLM serving process, the input of the SA block is first multiplied with three weight matrices W_Q , W_K , and W_V , yielding the outputs termed query (q), key (k), and value (v), respectively. The resulting q and k , in combination with v , will then undergo multiplication, softmax, and residual addition to generate the SA output. The output from the SA will then be passed to the FFN for further processing, which typically involves a standard MLP [63, 65] or gated MLP [45, 74, 75].

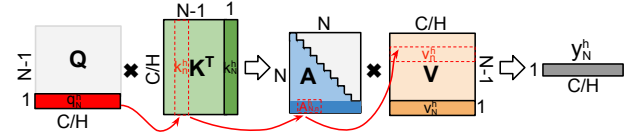


Figure 2: An example on KV vector computation.

The FFN consists of multiple fully connected (FC) layers along with an intermediate activation function, such as GeLU [31].

LLM serving involves two main stages: pre-filling and decoding. In the pre-filling stage, the model processes the context tokens in parallel. During the decoding stage, the model predicts the next token based on the current and previous tokens. This is done by combining the current input with information from previous tokens, expressed in terms of their Key and Value (KV) vectors. This process is repeated in an auto-regressive manner (Figure 1 (a)).

2.2 KV Caching

During the decoding stage, the KV vectors of each newly generated token are stored in a KV cache to enhance generation speed, as shown in Figure 1 (b). By doing so, it avoids recalculating vectors for earlier tokens with each new token generation. Specifically, to produce the output of an LLM block, the input vector of N th token, with a length C , is multiplied by W_Q , W_K , and W_V to generate three vectors: query q_N , key k_N , and value v_N , each with dimensions $1 \times C$, where C denotes the channel size, as shown in Figure 1 (b). q_N , together with other KV vectors are then split along the channel dimension into multiple parts, each has a dimension of $1 \times \frac{C}{H}$, where H denotes number of heads. The resulting vectors for head h are denoted as q_N^h , k_N^h , v_N^h , respectively. The KV vectors from the previous $N-1$ tokens are then loaded from memory. For each head h , the dot products are then performed between q_N^h and each of the key vectors k_n^h , $1 \leq n \leq N$, and the result is passed through a softmax function, yielding a vector of attention scores, denoted as A_N^h , with dimensions of $1 \times N$. Next, the attention score vector is used to compute a dot product with each of the value vectors v_n^h , $1 \leq n \leq N$, producing a result vector y_N^h of length $\frac{C}{H}$. y_N^h will then be concatenated across multiple heads, the resulting vector y_N which is further multiplied with W_O . The process is illustrated in Figure 2 and can be represented by the following equations:

$$A_N^h = \text{softmax}([q_N^{hT} k_1^h, q_N^{hT} k_2^h, \dots, q_N^{hT} k_N^h]) \quad (1)$$

$$y_N^h = \sum_{1 \leq n \leq N} (A_{N,n}^h \cdot v_n^h) \quad (2)$$

where $A_{N,n}^h$ represents the n -th element of A_N^h . As per Equation 1 and Equation 2, we notice that the relative order of KV vector pairs $[k_n^h, v_n^h]$ does not affect the decoding computation. In other words, if we swap the values of two pairs of KV vectors (e.g., swap $[k_1^h, v_1^h]$ with $[k_2^h, v_2^h]$), the result y_N^h produced using Equation 1 and 2 remains unchanged.

KV cache compression techniques can be broadly classified into two approaches: token dropping [27, 44, 47, 83, 88, 98] and KV cache quantization [32, 48, 82]. The token dropping strategy identifies

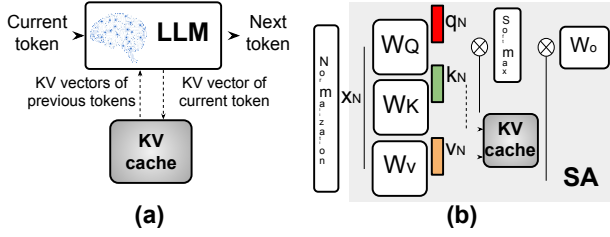


图 1: (a) LLM 代币生成。 (b) 用于中间数据存储的KV缓存, 其中N表示令牌索引。

然而, eDRAM 的一个主要缺点是需要定期刷新以防止数据因泄漏而丢失。具体来说, 刷新 eDRAM 单元需要读写操作, 从而增加延迟和功耗, 这会显著影响 LLM 的高效部署。为了解决集成 eDRAM 的挑战, 我们与基于 eDRAM 的硬件系统共同设计了 KV 缓存算法, 在不影响准确性的情况下实现高效的 KV 缓存实现。我们的贡献总结如下:

- 我们提出了 Kelle, 一种算法系统协同设计解决方案, 适用于在基于 eDRAM 的边缘系统上服务的设备内 LLM。为了优化 eDRAM 集成成本并提高 LLM 执行效率, 我们引入了基于注意力的驱逐和重新计算策略 (AERP) 和二维自适应刷新策略 (2DRP) 以实现高效的 KV 缓存实现 (第 4.1 和 4.2 节)。
- 我们设计了一款 Kelle 加速器, 它利用 eDRAM 作为主要片上存储, 具有定制的内存布局。为了最大限度地提高效率, 加速器集成了专用的 eDRAM 控制器和脉动驱逐器, 以实现高效的 AERP 和 2DRP 实施 (第 5 节)。
- 我们还介绍了 Kelle 调度程序 (第 6 节), 它采用高效的计算模式来优化 eDRAM 数据生命周期和 LLM 服务延迟, 从而显著减少 eDRAM 刷新能量和内存流量。
- 评估结果表明, 与其他基准硬件平台相比, Kelle 实现了 3.9× 加速和 4.5× 节能, 同时对 LLM 准确性的影响可以忽略不计 (第 7、8 节)。

2 背景及相关工作

2.1 大语言模型流程

现代 LLM (例如 Llama 系列 [74, 75]、GPT 系列 [11, 65]) 被构造为变压器解码器堆栈, 每个解码器包含两个基本组件: 自注意力 (SA) 块和前馈网络 (FFN)。在 LLM 服务过程中, SA 块的输入首先与三个权重矩阵 W_Q 、 W_K 和 W_V 相乘, 分别产生称为查询 (q)、键 (k) 和值 (v) 的输出。所得的 q 和 k 与 v 结合, 将进行乘法、softmax 和残差加法以生成 SA 输出。然后, SA 的输出将被传递到 FFN 进行进一步处理, 这通常涉及标准 MLP [63, 65] 或门控 MLP [45, 74, 75]。

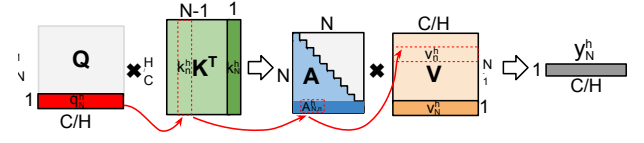


图 2: KV 向量计算的示例。

FFN 由多个全连接 (FC) 层以及中间激活函数 (例如 GeLU [31]) 组成。

LLM 服务涉及两个主要阶段: 预填充和解码。在预填充阶段, 模型并行处理上下文标记。在解码阶段, 模型根据当前和先前的标记预测下一个标记。这是通过将当前输入与先前令牌的信息相结合来完成的, 以它们的键和值 (KV) 向量表示。该过程以自回归方式重复 (图 1 (a))。

2.2 KV 缓存

在解码阶段, 每个新生成的令牌的 KV 向量都存储在 KV 缓存中以提高生成速度, 如图 1 (b) 所示。通过这样做, 它可以避免在每个新令牌生成时重新计算早期令牌的向量。具体来说, 为了生成 LLM 块的输出, 将长度为 C 的第 N 个令牌的输入向量乘以 W_Q 、 W_K 和 W_V 以生成三个向量: 查询 q_N 、键 k_N 和值 v_N , 每个向量的维度为 $1 \times C$, 其中 C 表示通道大小, 如图 1 所示(b)。然后, q_N 与其他 KV 向量一起沿着通道维度分成多个部分, 每个部分的维度为 $1 \times \frac{C}{H}$, 其中 H 表示头的数量。头 h 的结果向量分别表示为 q_N^h 、 k_N^h 、 v_N^h 。然后从内存中加载来自先前 $N-1$ 令牌的 KV 向量。对于每个头 h , 然后在 q_N^h 和每个键向量 $k_n^h, 1 \leq n \leq N-1$ 之间执行点积, 并将结果传递给 softmax 函数, 生成注意力分数向量, 表示为 A_N^h , 维度为 $1 \times N$ 。接下来, 使用注意力得分向量与每个值向量 $v_n^h, 1 \leq n \leq N-1$ 计算点积, 产生长度为 $\frac{C}{H}$ 的结果向量 y_N^h 。然后, y_N^h 将跨多个头连接, 所得向量 y_N 进一步与 W_O 相乘。该过程如图 2 所示, 可以用以下等式表示:

$$A_N^h = \text{softmax}([q_N^{h\top} k_1^h, q_N^{h\top} k_2^h, \dots, q_N^{h\top} k_N^h]) \quad (1)$$

$$y_N^h = \sum_{1 \leq n \leq N} (A_{N,n}^h \cdot v_n^h) \quad (2)$$

其中 $A_{N,n}^h$ 表示 A_N^h 的第 n 个元素。根据公式 1 和公式 2, 我们注意到 KV 向量对 $[k_n^h, v_n^h]$ 的相对顺序不会影响解码计算。换句话说, 如果我们交换两对 KV 向量的值 (例如, 将 $[k_1^h, v_1^h]$ 与 $[k_2^h, v_2^h]$ 交换), 则使用等式 1 和 2 生成的结果 y_N^h 保持不变。

KV 缓存压缩技术可以大致分为两种方法: 令牌丢弃 [27, 44, 47, 83, 88, 98] 和 KV 缓存量化 [32, 48, 82]。代币丢弃策略确定

Table 1: Comparison of SRAM and eDRAM.

65nm	Area	Access	Access	Leakage	Refresh	Retention
4MB		Latency	Energy	Power	Energy	Time
SRAM	7.3 mm ²	2.6ns	185.9 pJ/byte	415mW	NA	NA
eDRAM	3.2 mm ²	1.9ns	84.8 pJ/byte	154mW	1.14 mJ	45 μ s [38]

and permanently discards unimportant tokens, making them inaccessible thereafter. StreamLLM [83] identifies *sink tokens*, which are tokens at the start of a sequence that are critical for LLM performance, and preserves recent tokens to maintain performance. H2O [98] identifies *heavy hitter tokens* that have high accumulated attention scores. KIVI [49] groups the KV vectors channel-wise to achieve 2-bit asymmetric quantization. QuaRot [6] utilizes zero-shot Hadamard transformation to reduce the outliers in the model and enables 4-bit quantization. Speculative decoding [33, 34, 42, 51] present another inference technique that accelerates LLMs by using a lightweight draft model to propose multiple tokens, which are then selectively verified by the full model. Kelle can be adopted in orthogonal with the speculative decoding techniques.

2.3 Embedded DRAM

Various eDRAM circuit designs [28, 89] have emerged as alternatives to SRAM, with some requiring only two transistors. Among these, 3T-eDRAM stands out, offering over twice the density and reducing static power dissipation by $3.5\times$ [13, 17] compared to SRAM. Table 1 presents a comparison between 3T-eDRAM and SRAM, with results simulated using Destiny [60] on a 65nm technology node. The advantages of eDRAM, including higher storage density, lower access latency and energy, make it an attractive choice for LLM implementation. Although eDRAM provides several advantages, a serious drawback is the need for periodic refreshes to prevent data corruption caused by charge leakage. As a result, eDRAM is best suited for storing **large amounts of transient data**, where frequent refreshes can be avoided.

Research has explored using eDRAM in accelerator systems to facilitate CNN computation [15, 54, 76, 97], with proposed methods to mitigate refresh power overhead. DaDianNao [15] partitions eDRAM into banks to mitigate refresh failures but does not address the challenges of refresh energy consumption or data retention. RANA [76] injects the bit retention errors during the training process of CNN to mitigate the accuracy drop caused by low refresh frequency. CAMEL [97] optimizes CNN model architecture to shorten the data lifetime during training. While prior research has shown the efficiency of eDRAM in Convolutional Neural Network (CNN) inference and training, its potential has yet to be explored for LLMs. In contrast, Kelle focuses on minimizing the off-chip memory access of KV cache in LLMs using eDRAM, an area previously unexplored.

2.4 Edge LLM Accelerator

To enable deployment of LLMs on edge devices, several studies have proposed methods to improve the accuracy of quantized transformers [23, 30, 40, 46, 50, 57, 85, 92, 100]. Tender [40] suggests a hardware-efficient LLM quantization method by making the scale factor a power of two. COMET [46] designs efficient mixed-precision GPU kernels for 4-bit LLM quantization. Other works, such as FlexGen [68], InfiniGen [41], InstInfer [56], LLM.npu [86],

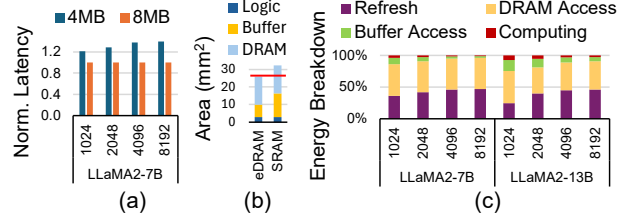


Figure 3: (a) Normalized latency of edge systems with 4MB vs. 8MB SRAM across models and sequence lengths. (b) Area breakdown of the edge systems with 8MB eDRAM and 8MB SRAM. Red line is the area budget. (c) Energy breakdown of the edge system integrating eDRAM. The decoding lengths are shown with a prefilling length of 512. An 8MB eDRAM is used to store KV cache for a subset of layers during decoding. The reported DRAM energy accounts for both model weight access and KV cache offloading from eDRAM.

explore the model offloading strategy between on-chip units and main storage for efficient LLM deployment in resource constraint devices. Cambricon-LLM [90] proposes a chiplet-based hybrid architecture with NPU and a dedicated NAND flash chip to enable efficient on-device inference.

3 Why Use eDRAM for LLMs on Edge Devices?

3.1 Benefits and Challenges of Expanding On-Chip Memory

As shown by earlier research [90, 93, 99], the speed of serving LLMs is significantly constrained by the bandwidth of off-chip memory. In particular, accessing the KV cache poses the most critical bottleneck during the decoding stage of LLMs [41, 98, 99]. A straightforward approach to minimize off-chip memory usage is to expand the on-chip SRAM size, which decreases expensive off-chip memory accesses and boosts system performance [15, 76]. To illustrate this, we evaluate the latencies of two edge computational systems with 4MB and 8MB of SRAM executing the LLaMA2-7B models across different sequence lengths. Tests are conducted on a simulated platform with a 32×32 systolic array for 8-bit MAC operations, and 16GB DRAM with 64GB/s bandwidth, reflective of an edge tensor processing unit (TPU) similar to the Google Coral edge device [72]. As shown in Figure 3 (a), doubling the SRAM size leads to an average of $1.27\times$ speedup. However, expanding the SRAM capacity from 4MB to 8MB in the evaluation platform increases the power consumption and chip area by 29% and 26%, respectively. Given the limited area and power budget for edge environment, increasing SRAM size reduces the resource available for other critical components, leading to a suboptimal system performance [14, 39, 79]. Therefore, we have the following observation:

Obs. 1: Larger on-chip memory alleviates the KV caching bottleneck in LLM, but brings area and power penalties in edge devices using SRAM as the on-chip storage.

表 1: SRAM 和 eDRAM 的比较。

65nm	Area	Access Latency	Access Energy	Leakage Power	Refresh Energy	Retention Time
4MB						
SRAM	7.3 mm ²	2.6ns	185.9 pJ/byte	415mW	NA	NA
eDRAM	3.2 mm ²	1.9ns	84.8 pJ/byte	154mW	1.14 mJ	45 μ s [38]

并永久丢弃不重要的令牌，使它们此后无法访问。StreamLLM [83] 识别接收器令牌，这些令牌是序列开头的令牌，对 LLM 性能至关重要，并保留最近的令牌以维持性能。H2O [98] 识别出具有高累积注意力分数的重量级代币。KIVI [49] 按通道对 KV 向量进行分组，以实现 2 位非对称量化。QuaRot [6] 利用零样本 Hadamard 变换来减少模型中的异常值并实现 4 位量化。推测性解码 [33,34,42,51] 提出了另一种推理技术，该技术通过使用轻量级草稿模型提出多个令牌来加速 LLM，然后由完整模型有选择地验证这些令牌。Kelle 可以与推测解码技术正交地采用。

2.3 嵌入式 DRAM

各种 eDRAM 电路设计 [28, 89] 已经作为 SRAM 的替代品出现，其中一些仅需要两个晶体管。其中，3T-eDRAM 脱颖而出，与 SRAM 相比，其密度提高了一倍以上，静态功耗降低了 3.5 $\times$ [13, 17]。表 1 展示了 3T-eDRAM 和 SRAM 之间的比较，以及在 65 nm 技术节点上使用 Destiny [60] 模拟的结果。eDRAM 的优势包括更高的存储密度、更低的访问延迟和能耗，使其成为 LLM 实施的有吸引力的选择。尽管 eDRAM 具有多种优点，但一个严重的缺点是需要定期刷新以防止电荷泄漏导致的数据损坏。因此，eDRAM 最适合存储大量瞬态数据，可以避免频繁刷新。

研究探索了在加速器系统中使用 eDRAM 来促进 CNN 计算 [15,54,76,97]，并提出了减轻刷新功率开销的方法。DaDianNa o [15] 将 eDRAM 划分为多个存储体以减轻刷新故障，但没有解决刷新能耗或数据保留的挑战。RANA[76]在CNN的训练过程中注入位保留误差，以减轻低刷新频率引起的精度下降。CAMEL [97]优化了 CNN 模型架构，以缩短训练期间的数据寿命。虽然之前的研究已经表明 eDRAM 在卷积神经网络 (CNN) 推理和训练中的效率，但其潜力尚未为**大语言模型** 相比之下，Kelle 专注于使用 eDRAM 最大限度地减少 LLM 中 KV 缓存的片外内存访问，这是一个以前未探索过的领域。

2.4 边缘大语言模型

为了能够在边缘设备上部署 LLM，一些研究提出了提高量化 Transformer 精度的方法 [23,30,40,46,50,57,85,92,100]。Tender [40] 通过将比例因子设置为 2 的幂，提出了一种硬件高效的 LLM 量化方法。COMET [46] 为 4 位 LLM 量化设计了高效的混合精度 GPU 内核。其他作品，如 FlexGen [68]、InfiniGen [41]、InstInfer [56]、LLM.npu [86]、

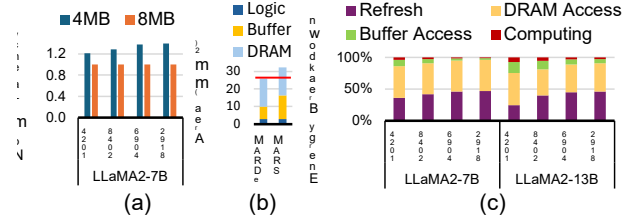


图 3: (a) 不同模型和序列长度的 4MB 与 8MB SRAM 边缘系统的归一化延迟。(b) 具有 8MB eDRAM 和 8MB SRAM 的边缘系统的面积细分。红线是面积预算。(c) 集成 eDRAM 的边缘系统的能量分解。解码长度显示为预填充长度 512。8MB eDRAM 用于在解码期间存储层子集的 KV 缓存。报告的 DRAM 能耗同时考虑了模型权重访问和 eDRAM 的 KV 缓存卸载。

探索片上单元和主存储之间的模型卸载策略，以在资源受限设备中高效部署 LLM。Cambricon-LLM [90]提出了一种基于小芯片的混合架构，配备 NPU 和专用 NAND 闪存芯片，以实现高效的设备上推理。

3 为什么在边缘设备上使用 eDRAM 进行大语言模型？

3.1 扩展片上存储器的好处和挑战

正如早期研究 [90,93,99] 所示，LLM 的服务速度很大程度上受到片外存储器带宽的限制。特别是，访问 KV 缓存在 LLM 的解码阶段构成了最关键的瓶颈[41,98,99]。最大限度地减少片外存储器使用的一个直接方法是扩大片上 SRAM 的大小，这可以减少昂贵的片外存储器访问并提高系统性能 [15, 76]。为了说明这一点，我们评估了两个具有 4MB 和 8MB SRAM 的边缘计算系统在不同序列长度上执行 LLaMA2-7B 模型的延迟。测试是在模拟平台上进行的，该平台具有用于 8 位 MAC 操作的 32×32 脉动阵列，以及具有 64GB/s 带宽的 16GB DRAM，反映了类似于 Google Coral 边缘设备 [72] 的边缘张量处理单元 (TPU)。如图 3 (a) 所示，SRAM 大小加倍可实现平均 1.27 $\times$ 加速。然而，将评估平台中的 SRAM 容量从 4MB 扩大到 8MB，功耗和芯片面积分别增加了 29% 和 26%。鉴于边缘环境的面积和功率预算有限，增加 SRAM 大小会减少其他关键组件的可用资源，从而导致系统性能不理想 [14,39,79]。因此，我们有以下观察：

观察。1: 更大的片上存储器缓解了 LLM 中的 KV 缓存瓶颈，但在使用 SRAM 作为片上存储的边缘设备中带来了面积和功耗损失。

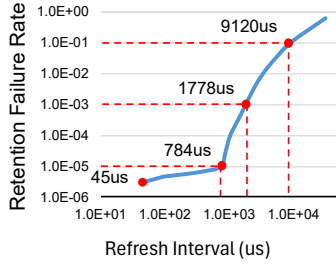


Figure 4: 65nm eDRAM retention failure distribution at 105°C [38].

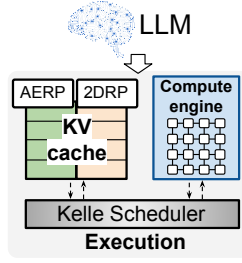


Figure 5: Overview of the Kelle accelerator.

3.2 Pros and Cons of Integrating eDRAM

To increase the on-chip memory size without increasing area, one approach is to replace SRAM with eDRAM. eDRAM not only provides more than twice the capacity under the same area of SRAM, but also consumes lower access and leakage energy according to Table 1. As shown in Figure 3 (b), the evaluation system with 8MB eDRAM takes less area than the system with 8MB SRAM, leading to lower LLM serving latency within a smaller chip size. Extensive research [5, 16, 84] and commercial products [22, 24, 28, 37, 80] demonstrate the feasibility of integrating eDRAM as the primary on-chip storage medium. However, its potential to benefit LLM serving in edge devices has not been explored. Although eDRAM offers several advantages, prior studies [76, 97] have shown that refresh operations can become a significant bottleneck in overall system energy consumption. Moreover, when eDRAM is used to store data with longer lifespans, infrequent refreshes can elevate the risk of readout errors, as illustrated in Figure 4. The retention failure rates are represented as the percentage of bits with retention errors, as the refresh interval varies. To illustrate this issue, we use an 8MB eDRAM to replace the 4MB SRAM in the system described in Section 3.1. The eDRAM refresh interval is set to 45 μ s to ensure no data corruption. We evaluate the energy consumption of the eDRAM system across different models and sequence lengths. As shown in Figure 3 (c), without optimization, eDRAM refresh operations take up to 46% of the total energy consumption, leading to 1.7 $\times$ more energy consumption on average.

Obs. 2: Under the same chip area, eDRAM can bring latency benefits over SRAM for LLM serving in edge devices. However, to fully leverage its power advantages, eDRAM refresh operations must be greatly minimized.

3.3 Kelle: Co-design KV Caching and eDRAM

To minimize eDRAM energy consumption, three effective strategies are: reducing data refresh frequency, decreasing stored data size, and decreasing data lifetime. To enable eDRAM for enhanced LLM serving performance in edge devices, we propose *Kelle*, a hardware and algorithm co-design solution for minimized eDRAM refresh energy and efficient KV cache management.

3.3.1 eDRAM Refresh Control. Reducing data refresh frequency may raise the risk of retention failures, causing data corruption. This leads to a key question: *To what extent can LLMs tolerate data*

corruption in the KV cache without compromising accuracy? Led by this question, we co-design the eDRAM memory layout and controller with *two-dimensional adaptive refresh policy* (2DRP), which sets fine-grained dynamic refresh intervals described in Section 4.2.

3.3.2 KV Cache Eviction. A smaller KV cache can significantly reduce the data storage demands on eDRAM, resulting in lower refresh energy consumption and improved system performance. Previous studies have observed that evicting unimportant tokens does not compromise the generation quality. However, to identify the unimportant tokens, previous works either require profiling of the sequences [27, 47], or extra computation [83, 98]. To manage the KV cache efficiently, we propose a novel *systolic evictor* architecture to accelerate the operation of *attention-based eviction and recomputation policy* (AERP), as described in Section 4.1.

3.3.3 KV Vector Recomputation. As the sequence length grows, the benefit of KV caching diminishes at a certain threshold since the time for accessing off-chip memory might outweigh that for recomputing partial KV tensors. Recomputation aligns well with the strength of eDRAM to store transient data, as shown in Section 4.1. However, the balance between recomputation and storage requires careful scheduling considering the hardware features. We propose the *Kelle Scheduler* to reduce the KV vector data lifetime via designing the computational pattern, which is depicted in Section 6.

4 Kelle Algorithm

In this section, we present the efficient algorithms used within the Kelle framework, with an overview provided in Figure 5. During execution, Kelle utilizes *attention-based eviction and recomputation policy* (AERP) and the *two-dimensional adaptive refresh policy* (2DRP) to manage eDRAM operation, as described in Section 4.1 and Section 4.2.

4.1 Attention-based Eviction and Recomputation Policy

We begin by discussing the eviction policy when the eDRAM capacity is reached during the decoding stage.

4.1.1 Eviction policy. For a KV cache with limited capacity, capable of holding up to N' tokens, during the decoding stage, the arrival of the $(N'+1)$ -th token requires the eviction of the KV vectors $[k_n^h, v_n^h]$ from one of the tokens n (where $1 \leq n \leq N'$). The KV vectors of the h -th head and n -th token to be evicted are selected based on their importance s_n^h , which is computed by summing the attention scores (Equation 1) with all other tokens in KV cache, shown as:

$$s_n^h = \sum_{1 \leq i \leq n} A_{n,i}^h \quad (3)$$

An example of the eviction process is illustrated in Figure 6. Assume the KV cache has a budget to store a total of $N' = 4$ vectors. We consider a case with three attention heads. For clarity, we only depict the computation for the first head and omit the head notation. When $[k_5, v_5]$ arrives, the importance scores are first computed using Equation 3, as shown in Figure 6 (a). The KV vectors of the token with the smallest importance score (third token) are then evicted, as depicted in Figure 6 (b). By leveraging the fact that the computation of y_N is unaffected by the relative order of the KV

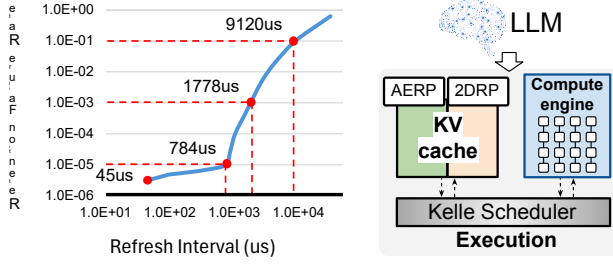


图 4: 105°C 时的 65nm eDRAM 保留故障分布[38]。图 5: Kelle 加速器概述。

3.2 集成 eDRAM 的优缺点

为了在不增加面积的情况下增加片上存储器的大小，一种方法是用 eDRAM 代替 SRAM。根据表1，eDRAM不仅在相同面积的SRAM下提供两倍以上容量，而且消耗更低的访问和泄漏能量。如图3(b)所示，采用8MB eDRAM的评估系统比采用8MB SRAM的系统占用更少的面积，从而在更小的芯片尺寸内实现更低的LLM服务延迟。广泛的研究[5,16,84]和商业产品[22,24,28,37,80]证明了集成eDRAM作为主要片上存储介质的可行性。然而，其在边缘设备中使大语言模型的潜力尚未得到探索。尽管 eDRAM 具有多种优势，但先前的研究 [76, 97] 表明刷新操作可能成为整个系统能耗的重要瓶颈。此外，当使用 eDRAM 存储寿命较长的数据时，不频繁的刷新会增加读出错误的风险，如图 4 所示。随着刷新间隔的变化，保留故障率表示为存在保留错误的位的百分比。为了说明这个问题，我们在第 3.1 节中描述的系统中使用 8MB eDRAM 替换 4MB SRAM。eDRAM 刷新间隔设置为 45μs 以确保不会损坏数据。我们评估了不同型号和序列长度的 eDRAM 系统的能耗。如图 3 (c) 所示，在没有优化的情况下，eDRAM 刷新操作占据总能耗的 46%，导致能耗平均增加 1.7×。

观察。2: 在相同的芯片面积下，eDRAM 可以为边缘设备中的 LLM 服务带来优于 SRAM 的延迟优势。然而，为了充分利用其功耗优势，必须大大减少 eDRAM 刷新操作。

3.3 Kelle: 协同设计KV Caching和eDRAM

为了最大限度地减少 eDRAM 能耗，三种有效的策略是：降低数据刷新频率、减小存储数据大小和缩短数据寿命。为了使 eDRAM 能够在边缘设备中增强 LLM 服务性能，我们提出了 Kelle，这是一种硬件和算法协同设计解决方案，可最大限度地减少 eDRAM 刷新能耗和高效的 KV 缓存管理。

3.3.1 eDRAM 刷新控制。降低数据刷新频率可能会增加保留失败的风险，从而导致数据损坏。这就引出了一个关键问题：法学硕士可以在多大程度上容忍数据

KV 缓存损坏而不影响准确性？在这个问题的引导下，我们利用二维自适应刷新策略（2DRP）共同设计了 eDRAM 内存布局和控制策略，该策略设置了第 4.2 节中描述的细粒度动态刷新间隔。

3.3.2 KV 缓存驱逐。较小的KV缓存可以显著降低对eDRAM的数据存储需求，从而降低刷新能耗并提高系统性能。之前的研究发现，驱逐不重要的代币不会影响生成质量。然而，为了识别不重要的标记，以前的工作要么需要对序列进行分析 [27, 47]，要么需要额外的计算 [83, 98]。为了有效地管理 KV 缓存，我们提出了一种新颖的脉动驱逐器架构，以加速基于注意力的驱逐和重新计算策略（AERP）的运行，如第 4.1 节所述。

3.3.3 KV向量重新计算。随着序列长度的增长，KV 缓存的好处在一定阈值下会减弱，因为访问片外存储器的时间可能超过重新计算部分 KV 张量的时间。重新计算与 eDRAM 存储瞬态数据的能力非常契合，如第 4.1 节所示。然而，重新计算和存储之间的平衡需要考虑硬件特性来仔细调度。我们建议 Kelle Scheduler 通过设计计算模式来减少 KV 向量数据寿命，如第 6 节所示。

4 凯勒算法

在本节中，我们介绍 Kelle 框架中使用的高效算法，概述如图 5 所示。在执行过程中，Kelle 利用基于注意力的驱逐和重新计算策略（AERP）和二维自适应刷新策略（2DRP）来管理 eDRAM 操作，如第 4.1 节和第 4.2 节所述。

4.1 基于注意力的驱逐和重计算政策

我们首先讨论在解码阶段达到 eDRAM 容量时的驱逐策略。

4.1.1 驱逐政策。对于容量有限、最多能够容纳 N' 个令牌的 KV 缓存，在解码阶段，第 $(N'+1)$ 个令牌的到来需要从令牌 n （其中 $1 \leq n \leq N'$ ）之一驱逐 KV 向量 $[k_n^h, v_n^h]$ 。要驱逐的第 h 个头和第 n 个令牌的 KV 向量是根据其重要性 s_n^h 选择的，其重要性是通过将注意力分数（公式 1）与 KV 缓存中的所有其他令牌相加来计算的，如下所示：

$$s_n^h = \sum_{1 \leq i \leq N'} A_{n,i}^h \quad (3)$$

图 6 展示了逐出过程的示例。假设 KV 缓存有预算来存储总共 $N' = 4$ 个向量。我们考虑一个具有三个注意力头的情况。为了清楚起见，我们只描述第一个头的计算并省略头符号。当 $[k_5, v_5]$ 到达时，首先使用公式 3 计算重要性分数，如图 6 (a) 所示。然后，重要性得分最小的令牌（第三个令牌）的 KV 向量将被驱逐，如图 6 (b) 所示。利用 y_N 的计算不受 KV 的相对顺序影响的事实

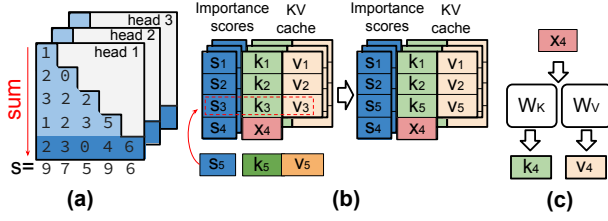


Figure 6: (a) Computation of the importance scores for each of three heads. (b) The KV vectors of the token with the lowest score is replaced with the new KV vectors. Input vector x_4 is stored because the fourth token is important among two out of three heads. Storing x_4 frees up an eDRAM entry, thereby reducing the eDRAM refresh cost. (c) Recompute KV vectors of the fourth token for saving eDRAM storage.

vectors, Equation 1 and Equation 2 can be computed by reading the KV vectors from the cache in sequence, without concern for their original token indices. It is important to note that the importance score s_n^h of the same token n might vary across different attention heads. As a result, the eviction pattern of KV vectors will differ across these heads h .

For the pre-filling stage with a context token length of N_{cxt} , all the context tokens are processed in parallel. For each head within each layer, importance scores of N th token are calculated as $s_N^h = \sum_{1 \leq n \leq N_{cxt}} A_{n,N}^h$. The tokens with top N' highest s_n^h will be retained in the KV cache for decoding operations.

In addition to the tokens with the highest s_n^h scores, the initial tokens and most recent tokens are also retained due to their proven impact on model performance, as demonstrated by prior work [83, 98] and supported by our experiments.

4.1.2 Recomputation policy. As discussed in Section 2.3, eDRAM is well-suited for storing transient data. Although the eviction policy reduces the number of KV vectors that need to be retained during model execution, storing KV vectors with long lifetime in eDRAM still incurs a serious cost due to the required refresh operations. To mitigate the refresh cost, we can further apply the recomputation technique. Specifically, for a subset of tokens N_{recomp} in the KV cache, their KV vectors will be recomputed using the corresponding input vector x_N , which serve as inputs to W_Q , W_K , and W_V , as depicted in Figure 1 (b). By leveraging recomputation, the storage requirement can be reduced from maintaining two vectors (K and V) to holding just a single vector (input x). This approach allows for K and V to be recomputed as needed, effectively mitigating the long data lifetime of KV vectors.

During decoding stage execution, the KV vectors k_N^h and v_N^h for all heads $h \in H$ are first recomputed by multiplying the input vector x_N with W_K and W_V (Figure 6 (c)), which are then used for the decoding process. To achieve savings in KV cache storage through recomputation, the storage cost of the input vector x_N , which is $1 \times C$, must be smaller than that of the recomputed KV vectors. To satisfy this, the KV vectors for a token N are recomputed using x_N if they would otherwise be retained in at least $\theta > 50\%$ of the heads without recomputation, where θ represents the **popularity** of the token. This approach is justified because the storage cost

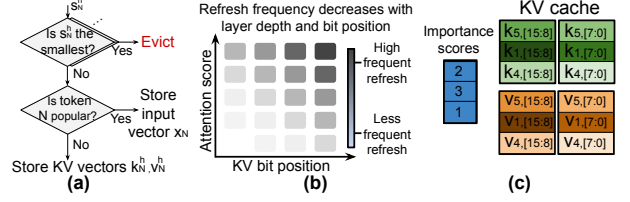


Figure 7: (a) Summary of AERP, only one head h is shown for simplicity. (b) 2D-adaptive refresh policy. (c) As an example on 2DRP. $k_5[15 : 8]$ denotes the eights to fifteenth bits of the key vector for the fifth token. Darker colors mean bits refreshed more frequently, resulting in a lower retention error rate.

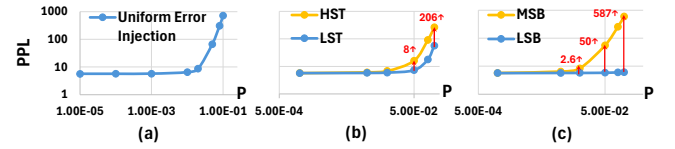


Figure 8: (a) PPLs with bit-flip error rates P . (b) LLM accuracy under varying bit-flip error rates when applying the bit flipping solely on (a) HST vs. LST and (b) MSB vs. LSB, where P denotes the error rate. A lower PPL reflects better performance, with the red numbers representing the gap between the PPL values.

associated with the KV vectors, calculated as $2 \times \frac{C}{H} \times \theta H$, would exceed the size of x_N (i.e., C). As shown in Figure 6 (b), the fourth token is deemed popular in two of three heads, so the input vector x_4 is retained, avoiding the storage of KV vectors.

In addition to storage savings, the recomputed KV vectors will be transient, as they are only used for a short duration to compute Equation 1, which further capitalizes on the advantages of eDRAM. Moreover, the additional cost of recomputation will be minimal due to the systolic array architecture for the computational engine, discussed in Section 5.2.

For the pre-filling stage, the importance scores s_n^h for each token n in head h are calculated first. Next, for each head h , KV vectors are evicted based on the importance scores of the corresponding tokens. Among tokens with high importance scores, those whose KV vectors are retained in at least 50% of the heads (i.e., popular tokens) have their input vectors x_n stored; otherwise, the KV vectors are stored. During decoding, each new token's storage format is dynamically determined by computing the popularity θ . Figure 7 (a) summarizes the overall AERP scheme. Although token popularity can vary during the decoding process, empirical evidence shows limited fluctuation, namely tokens important to over 50% of heads rarely decrease in importance. Therefore, in Kelle, once a token is stored with its input vectors, its storage format remains fixed throughout decoding unless it gets evicted.

4.2 Two-Dimensional Adaptive Refresh Policy

To explore the tolerance of LLMs to data corruption in the KV cache without compromising accuracy, we simulate retention failures by

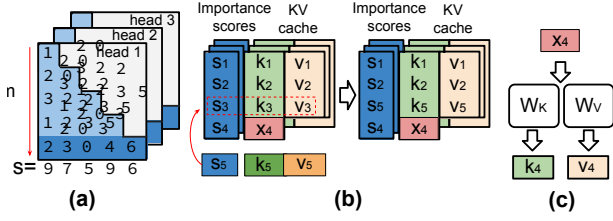


图 6: (a) 计算三个头中每个头的重要性分数。 (b) 得分最低的 token 的 KV 向量被新的 KV 向量替换。存储输入向量 x_4 是因为第四个标记在三个头中的两个中很重要。存储 x_4 可释放 eDRAM 条目, 从而降低 eDRAM 刷新成本。 (c) 重新计算第四个令牌的 KV 向量以节省 eDRAM 存储。

向量, 方程1和方程2可以通过按顺序从缓存中读取KV向量来计算, 而不用关心它们的原始令牌索引。值得注意的是, 同一 token n 的重要性得分 s_n^h 在不同的注意力头 h 上会有所不同。因此, KV 向量的逐出模式在这些头 h 上会有所不同。

对于上下文令牌长度为 N_{cxt} 的预填充阶段, 所有上下文令牌都是并行处理的。对于每层中的每个头, 第 N 个令牌的重要性分数计算为 $s_N^h = \sum_{1 \leq n \leq N_{cxt}} A_{n,N}^h$ 。具有 top N' 最高 s_n^h 的 token 将保留在 KV 缓存中用于解码操作。

除了 s_n^h 分数最高的标记之外, 初始标记和最近的标记也被保留, 因为它们对模型性能的影响已被证明, 如先前的工作 [83, 98] 所证明的并得到我们的实验的支持。

4.1.2 重新计算策略。如第 2.3 节所述, eDRAM 非常适合存储瞬态数据。尽管逐出策略减少了模型执行期间需要保留的 KV 向量的数量, 但由于需要刷新操作, 在 eDRAM 中存储具有较长生命周期的 KV 向量仍然会产生严重的成本。为了降低刷新成本, 我们可以进一步应用重新计算技术。具体来说, 对于 KV 缓存中的令牌子集 N_{recomp} , 它们的 KV 向量将使用相应的输入向量 x_N 重新计算, 该输入向量作为 W_Q , W_K , 和 W_V 的输入, 如图 1 (b) 所示。通过利用重新计算, 存储要求可以从维护两个向量 (K 和 V) 减少到仅保存一个向量 (输入 x)。这种方法允许根据需要重新计算 K 和 V, 从而有效地缓解 KV 向量的长数据生命周期。

在解码阶段执行期间, 首先通过将输入向量 x_N 与 W_K 和 W_V (图 6 (c)) 相乘来重新计算所有头 $h \in H$ 的 KV 向量 k_N^h 和 v_N^h , 然后将其用于解码过程。为了通过重新计算实现 KV 缓存存储的节省, 输入向量 x_N 的存储成本 (即 $1 \times C$) 必须小于重新计算的 KV 向量的存储成本。为了满足这一点, 如果令牌 N 的 KV 向量将保留在至少 $\theta > 50\%$ 的头中而无需重新计算, 则使用 x_N 重新计算, 其中 θ 表示令牌的流行度。这种方法是合理的, 因为存储成本

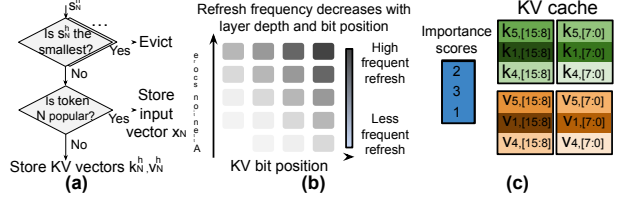


图 7: (a) AERP 概要, 为简单起见, 仅显示了一个头 h 。 (b) 2D 自适应刷新策略。 (c) 作为 2DRP 的示例。 $k_5[15:8]$ 表示第五个令牌的密钥向量的第八到第十五位。颜色越深意味着位刷新越频繁, 从而保留错误率越低。

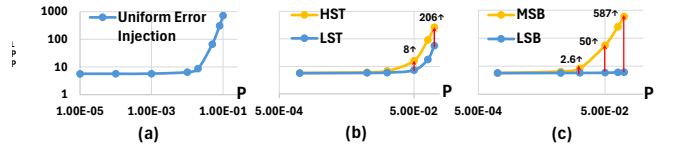


图 8: (a) 具有位翻转错误率 P 的 PPL。 (b) 仅在 (a) HST 与 LST 和 (b) MSB 与 LSB 上应用位翻转时, 不同位翻转错误率下的 LLM 精度, 其中 P 表示错误率。较低的 PPL 反映了更好的性能, 红色数字代表 PPL 值之间的差距。

与 KV 向量关联的值 (计算为 $2 \times \frac{C}{H} \times \theta H$) 将超过 x_N (即 C) 的大小。如图 6(b) 所示, 第四个 token 在三个头中的两个中被认为是受欢迎的, 因此保留输入向量 x_4 , 避免存储 KV 向量。

除了节省存储空间之外, 重新计算的 KV 向量将是暂时的, 因为它们仅在短时间内用于计算公式 1, 这进一步利用了 eDRAM 的优势。此外, 由于计算引擎的脉动阵列架构 (第 5.2 节中讨论), 重新计算的额外成本将是最小的。

对于预填充阶段, 首先计算头 h 中每个标记 n 的重要性得分 s_n^h 。接下来, 对于每个头 h , 根据相应令牌的重要性分数驱逐 KV 向量。在重要性得分较高的 token 中, KV 向量保留在至少 50% 的头部中的 token (即流行 token) 存储其输入向量 x_n ; 否则, 存储 KV 向量。在解码过程中, 每个新令牌的存储格式是通过计算流行度 θ 动态确定的。图 7 (a) 总结了整个 AERP 方案。尽管令牌流行度在解码过程中可能会有所不同, 但经验证据表明波动有限, 即对超过 50% 的人来说重要的令牌很少会降低重要性。因此, 在 Kelle 中, 一旦令牌与其输入向量一起存储, 其存储格式在整个解码过程中保持固定, 除非它被驱逐。

4.2 二维自适应刷新策略

为了探索 LLM 对 KV 缓存中数据损坏的容忍度而不影响准确性, 我们通过以下方式模拟保留失败

introducing bit flip errors across the eDRAM memory cells. Specifically, we assess the impact on the perplexity (PPL) of LLaMA2-7B models using the Wikitext-2 [52] dataset. Lower PPL indicates better performance. During execution, the bit flip errors are introduced in the KV cache with a uniform probability. The results, presented in Figure 8 (a), reveal that for error rates below 10^{-3} , the increase in PPL remains minimal, staying under 0.1. However, as the bit flip error continues to rise, PPL increases significantly. This suggests that LLMs can tolerate a certain degree of KV cache errors. A natural follow-up question arises: *Is it possible to develop a finer-grained refresh policy that could support even lower refresh frequencies while maintaining accuracy?*

In Section 4.1, tokens are evicted based on their importance scores, as defined in Equation 3. We hypothesize that a similar approach could be applied to the eDRAM refresh policy, assigning lower refresh frequencies to KV vectors or input vectors of less important tokens and higher frequency to those of more important tokens. To test this hypothesis, we implement an adaptive refresh policy and repeated the experiment. Tokens were divided into two groups based on their importance scores, referred to as the high score token (HST) group and the low score token (LST) group for simplicity. A probability p of bit retention failure (bit flip error) in the KV vectors was applied separately to the KV vectors of the corresponding tokens in HST and LST groups. The results in Figure 8 (b) show that LLM performance degrades more when bit retention failures affect the HST group than the LST group, indicating that tokens in the HST group require higher refresh frequency, thus supporting our hypothesis.

Additionally, it is reasonable to hypothesize that the less significant bits (LSBs) are less vulnerable to retention failure errors than the more significant bits (MSBs), as bit flip error on LSBs causes smaller changes in value. For each value in a KV vector, we introduce bit retention errors to either the MSBs (bits 15-8) or the LSBs (bits 7-0). The results, shown in Figure 8 (c), reveal that the MSBs are more sensitive to retention errors than the LSBs under the same bit flip error rate, further supporting our hypothesis.

Based on observations above, we propose an adaptive refresh control strategy called the *two-dimensional adaptive refresh policy* (2DRP), as shown in Figure 7 (b). This strategy adjusts the refresh frequency of each eDRAM cell based on both the bit position within each value in KV vectors or input vectors and the importance score of each token. An example of 2DRP is shown in Figure 7 (c), where the KV cache holds up to $N' = 3$ tokens. The refresh frequency increases with both the token importance and the significance of bit positions. During execution, the importance scores of KV and input vectors are dynamically calculated, and a refresh frequency is assigned accordingly based on these scores and bit positions.

5 Kelle Edge Accelerator

Figure 9 provides an overview of the Kelle accelerator. It incorporates a hybrid eDRAM-SRAM memory subsystem, a reconfigurable systolic array (RSA), and specialized function units (SFUs). The weights are quantized to 8 bits, and activations and KV vectors are maintained in 16 bits, with weights stored in SRAM, while activations and the KV vectors are held in eDRAM. During operation, systolic evictor accumulates attention scores and eDRAM

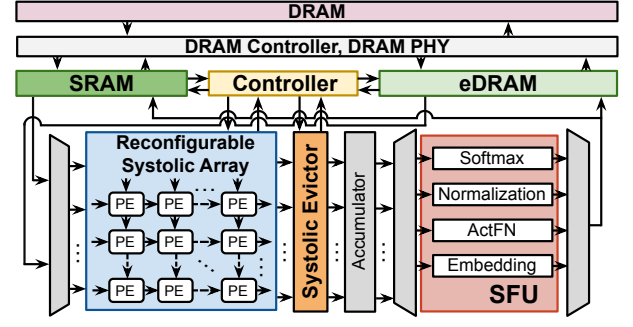


Figure 9: An overview of Kelle hardware accelerator.

controller handles KV vector eviction and recomputation while dynamically adjusting the refresh frequency, as discussed in Sections 4.1 and 4.2. Each processing element (PE) in RSA performs 8-bit multiply-accumulate (MAC) operations.

SFU handles non-linear operations, including activation functions, softmax, normalization, and positional embeddings. As prior research has shown [20, 62, 78, 81], the energy consumption of non-linear operations increases with input sequence length. Among these operations, softmax consumes significant resources. We employ online max calculation from Softmax [71] to minimize memory access. For other non-linear operations, we follow the computation flow and use lookup tables (LUTs) to perform the calculation.

5.1 Memory Subsystem

Figure 10 illustrates the memory subsystem of the Kelle accelerator. In this design, a 2MB SRAM stores the weights, while activations and KV vectors are held in a 256KB *activation eDRAM* and 4MB *KV cache eDRAM*, respectively. Kelle accelerator implements 2DRP by dividing KV vectors into four groups based on importance scores and bit positions and applies the refresh frequency accordingly. Specifically, the MSBs (bits 15-8) of the KV vectors for tokens in the HST group are refreshed at the highest, while LSBs (bits 7-0) of the KV vectors for tokens in the LST group are refreshed at the lowest frequency. To support the AERP, for certain tokens, input vectors are stored in the KV cache eDRAM instead of KV vectors. These input vectors are then divided into four groups and controlled in the same manner as KV vectors, organized by importance scores and bitwidth. For simplicity, we use KV vectors to describe the memory subsystem design, without referencing input vectors.

To execute 2DRP during LLM inference, each element of the KV vectors is split bitwise and stored across different eDRAM banks. Specifically, for KV vectors, the MSBs and LSBs are stored in separate KV cache eDRAM banks, referred to as *MSB banks* and *LSB banks*, highlighted by the darker and lighter colors in Figure 10. The importance score of each token is computed dynamically using Equation 3 under 4-bit precision and stored in a register file, with each entry corresponding to a KV vector spanning four banks. KV vectors corresponding to the same token share the same address across different eDRAM banks. The system features a single eviction controller that manages AERP across all four banks, along with two refresh controllers responsible for executing 2DRP separately over MSB and LSB banks, respectively.

在 eDRAM 存储单元中引入位翻转错误。具体来说，我们使用 Wikitext-2 [52] 数据集评估对 LLaMA2-7B 模型的困惑度 (PPL) 的影响。PPL 越低表明性能越好。在执行过程中，KV 缓存中会以均匀的概率引入位翻转错误。图 8 (a) 中的结果表明，当错误率低于 10^{-3} 时，PPL 的增加仍然很小，保持在 0.1 以下。然而，随着位翻转错误继续上升，PPL 显著增加。这表明 LLM 可以容忍一定程度的 KV 缓存错误。一个自然的后续问题出现了：是否有可能开发一种更细粒度的刷新策略，在保持准确性的同时支持更低的刷新频率？

在第 4.1 节中，令牌根据其重要性分数被逐出，如公式 3 中所定义。我们假设类似的方法可以应用于 eDRAM 刷新策略，为不太重要的令牌的 KV 向量或输入向量分配较低的刷新频率，为较重要的令牌分配较高的频率。为了检验这个假设，我们实施了自适应刷新策略并重复了实验。令牌根据其重要性分数分为两组，为简单起见，称为高分令牌 (HST) 组和低分令牌 (LST) 组。KV 向量中的位保留失败 (位翻转错误) 的概率 p 被分别应用于 HST 和 LST 组中的相应令牌的 KV 向量。图 8 (b) 中的结果表明，当位保留故障影响 HST 组而不是 LST 组时，LLM 性能下降更多，表明 HST 组中的令牌需要更高的刷新频率，从而支持我们的假设。

此外，可以合理地假设，较低有效位 (LSB) 比较高有效位 (MSB) 更不容易受到保留失败错误的影响，因为 LSB 上的位翻转错误导致值的变化较小。对于 KV 向量中的每个值，我们向 MSB (位 15-8) 或 LSB (位 7-0) 引入位保留错误。结果如图 8 (c) 所示，表明在相同的位翻转错误率下，MSB 对保留错误比 LSB 更敏感，进一步支持了我们的假设。

基于上述观察，我们提出了一种自适应刷新控制策略，称为二维自适应刷新策略 (2DRP)，如图 7 (b) 所示。该策略根据 KV 向量或输入向量中每个值内的位位置以及每个令牌的重要性得分来调整每个 eDRAM 单元的刷新频率。2DRP 的示例如图 7 (c) 所示，其中 KV 缓存最多可容纳 $N' = 3$ 个令牌。刷新频率随着令牌重要性和位位置的重要性而增加。在执行过程中，动态计算 KV 和输入向量的重要性分数，并根据这些分数和位位置相应地分配刷新频率。

5 凯勒边缘加速器

图 9 提供了 Kelle 加速器的概述。它集成了混合 eDRAM-SRAM 内存子系统、可重构脉动阵列 (RSA) 和专用功能单元 (SFU)。权重被量化为 8 位，激活和 KV 向量保持在 16 位，权重存储在 SRAM 中，而激活和 KV 向量保存在 eDRAM 中。在操作过程中，收缩驱动器会累积注意力分数和 eDRAM

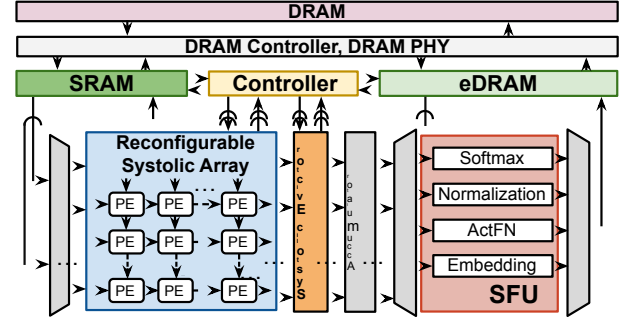


图 9: Kelle 硬件加速器概述。

控制器处理 KV 向量驱逐和重新计算，同时动态调整刷新频率，如第 4.1 节和第 4.2 节所述。RSA 中的每个处理元件 (PE) 执行 8 位乘法累加 (MAC) 运算。

SFU 处理非线性操作，包括激活函数、softmax、归一化和位置嵌入。正如先前的研究所示 [20, 62, 78, 81]，非线性操作的能耗随着输入序列长度的增加而增加。在这些操作中，softmax 消耗了大量资源。我们采用 Softmax [71] 的在线最大值计算来最小化内存访问。对于其他非线性运算，我们遵循计算流程并使用查找表 (LUT) 来执行计算。

5.1 内存子系统

图 10 说明了 Kelle 加速器的内存子系统。在此设计中，2MB SRAM 存储权重，而激活和 KV 向量分别保存在 256KB 激活 eDRAM 和 4MB KV 缓存 eDRAM 中。Kelle 加速器通过根据重要性分数和位位置将 KV 向量分为四组并相应地应用刷新频率来实现 2DRP。具体地，HST 组中令牌的 KV 向量的 MSB (比特 15-8) 刷新频率最高，而 LST 组中令牌的 KV 向量的 LSB (比特 7-0) 刷新频率最低。为了支持 AERP，对于某些令牌，输入向量存储在 KV 缓存 eDRAM 中，而不是 KV 向量。然后，这些输入向量被分为四组，并以与 KV 向量相同的方式进行控制，按重要性分数和位宽进行组织。为了简单起见，我们使用 KV 向量来描述内存子系统设计，而不引用输入向量。

为了在 LLM 推理期间执行 2DRP，KV 向量的每个元素都按位分割并存储在不同的 eDRAM Bank 中。具体来说，对于 KV 向量，MSB 和 LSB 存储在单独的 KV 缓存 eDRAM 存储体中，称为 MSB 存储体和 LSB 存储体，在图 10 中以较深和较浅的颜色突出显示。每个标记的重要性分数是使用公式 3 在 4 位精度下动态计算的，并存储在寄存器文件中，每个条目对应于跨越四个存储体的 KV 向量。对应于同一令牌的 KV 向量在不同的 eDRAM Bank 之间共享相同的地址。该系统具有一个单一驱逐控制器，用于管理所有四个存储体的 AERP，以及两个刷新控制器，分别负责在 MSB 和 LSB 存储体上分别执行 2DRP。

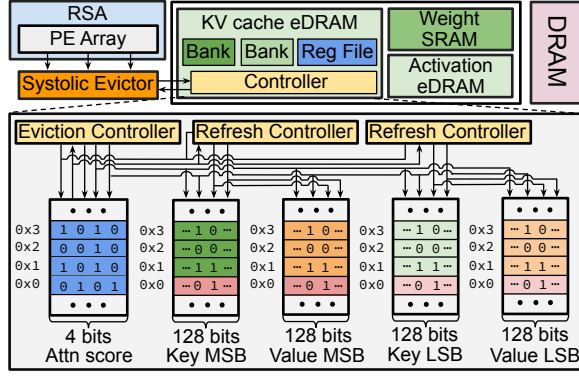


Figure 10: Kelle memory subsystem. Input vectors of certain tokens are stored in the KV cache, represented by red rows.

In each MSB and LSB bank, the tokens are further divided into two groups according to their attention scores, with a counter in the refresh controller monitoring each group’s refresh interval. The controller iterates through the eDRAM entries, identifying the group of each token by reading its attention score from the register file. When the refresh interval for a specific group expires, the controller triggers a *refresh* signal. The corresponding KV vector addresses for the tokens in that group are then computed, and the KV vectors are read out and written back as part of the refresh procedure. The refresh operation is triggered when the KV vectors are not used by the model, so the refresh latency can be hidden. When the KV cache reaches capacity and a new token arrives, the eviction controller receives the evict token index from the systolic evictor and replaces that token with the new token.

To fully utilize the 32×32 RSA by feeding data in parallel and avoiding bank conflicts, the Kelle KV cache is divided into 32 banks. Specifically, 8 banks are assigned to each of the Key MSB, Key LSB, Value MSB, and Value LSB groups. With this design and pipelined cache read, Kelle eDRAM provides sufficient bandwidth to make full use of the RSA without bank conflicts. Additionally, other eDRAM access operations such as token read and token eviction operate independently, this effectively mitigates bank conflicts.

During the LLM execution, the RSA I/O controller efficiently reconstructs the data from different banks for computation with minimal overhead. Additionally, the Kelle accelerator stores the KV vectors from a subset of LLM layers in eDRAM, with the number of layers determined by the specific LLM size and text length. eDRAM greatly minimizes off-chip memory access overhead.

5.2 Reconfigurable Systolic Array

The systolic array core consists of a 32×32 two-dimensional array that processes inputs in a staggered manner, sending the computed partial sums to the accumulator and SFUs. It utilizes a weight-stationary data flow, as shown in Figure 11 (a). We employ a reconfigurable strategy similar to that in FAST [96] to perform in-place transposed matrix multiplication.

Importantly, the recomputation in Section 4.1 introduces minimal overhead in the LLM decoding stage. Leveraging the strength of systolic arrays for matrix operations, the recomputed token vectors

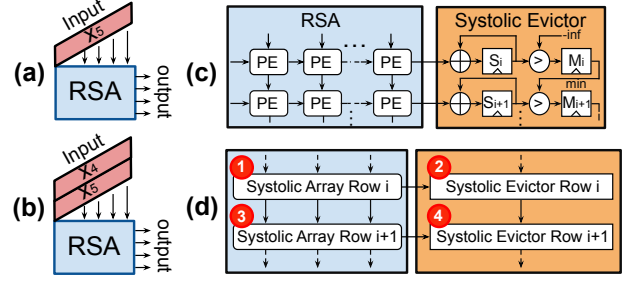


Figure 11: (a) and (b) show the impact of recomputation on RSA operation. (c) The integration of RSA and systolic evictor. (d) The execution order of systolic array and systolic evictor. Numbers in red circles denote the orders.

can be combined with the current token’s input vector to create an input matrix efficiently. Using the same notation as the example in Figure 6 (b), Figure 11 (a) shows the input vector x_5 of the current token being sent to RSA for KV vector computation. To recompute KV vectors for the fourth token, x_4 and x_5 can be combined into a matrix, causing minimal latency and energy growth in Figure 11(b).

5.3 Systolic Evictor

The token eviction process in the AERP algorithm includes computing the attention score as described in Equation 1, updating the importance score based on Equation 3, identifying the token with the lowest importance score, and performing the KV cache update.

To efficiently implement the eviction algorithm, we propose a systolic evictor (SE) which operates in a systolic style and is integrated with the RSA to search the minimum importance score on-the-fly. The importance score is calculated by summing the QK^T results in Equation 1 without passing through the softmax. This integration ensures the token with the minimum importance score is found as soon as the new token’s attention score is calculated from the RSA. After finding the index of the token with minimum importance score, the SE sends the index to the eviction controller in the eDRAM controller to evict the corresponding token. Figure 11 (c) illustrates the design of SE and its integration in the RSA. The SE comprises a column of registers, denoted as S in Figure 11 (c), to preload the importance scores of previous tokens, and a register chain, denoted as M , periodically propagates the minimum importance score ($\min$) from top to bottom. Figure 11 (d) illustrates execution order of the RSA and SE. In one cycle, the attention score is calculated from the i -th row of RSA, and then the i -th row of SE updates the importance score and the minimum importance score index, marked as Step 1 and Step 2, respectively. In the next cycle, the same operations are executed in the next row of RSA and SE, marked as Step 3 and Step 4, respectively. The systolic evictor avoids the extra LLM execution latency from the minimum search.

6 Kelle Scheduler

To further minimize eDRAM refresh energy, we introduce a novel computation pattern that shortens data lifetime and accelerates LLM inference, all without compromising accuracy.

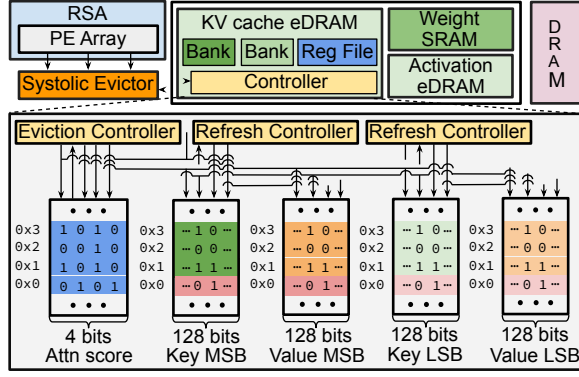


图 10: Kelle 内存子系统。某些 token 的输入向量存储在 KV 缓存中，用红色行表示。

在每个 MSB 和 LSB 存储体中，令牌根据其注意力分数进一步分为两组，刷新控制器中的计数器监视每组的刷新间隔。控制器迭代 eDRAM 条目，通过从寄存器文件中读取其注意力分数来识别每个令牌的组。当特定组的刷新间隔到期时，控制器触发刷新信号。然后计算该组中令牌的相应 KV 向量地址，并且作为刷新过程的一部分读出并写回 KV 向量。当模型不使用 KV 向量时会触发刷新操作，因此可以隐藏刷新延迟。当 KV 缓存达到容量并且新令牌到达时，逐出控制器从脉动逐出器接收逐出令牌索引，并用新令牌替换该令牌。

为了通过并行馈送数据并避免存储体冲突来充分利用 32×32 RSA，Kelle KV 缓存分为 32 个存储体。具体地，8 个存储体被分配给每个 Key MSB、Key LSB、Value MSB 和 Value LSB 组。通过这种设计和流水线缓存读取，Kelle eDRAM 提供了足够的带宽来充分利用 RSA，而不会发生存储体冲突。此外，其他 eDRAM 访问操作（例如令牌读取和令牌驱逐）独立运行，这有效地缓解了存储体冲突。

在 LLM 执行期间，RSA I/O 控制器有效地重建来自不同存储体的数据，以最小的开销进行计算。此外，Kelle 加速器将来自 LLM 层子集的 KV 向量存储在 eDRAM 中，层数由特定的 LLM 大小和文本长度决定。eDRAM 极大地减少了片外内存访问开销。

5.2 可重构脉动阵列

脉动阵列核心由 32×32 二维阵列组成，该阵列以交错方式处理输入，将计算出的部分和发送到累加器和 SFU。它利用重量固定数据流，如图 11 (a) 所示。我们采用类似于 FAST [96] 的可重构策略来执行就地转置矩阵乘法。

重要的是，4.1 节中的重新计算在 LLM 解码阶段引入了最小的开销。利用脉动数组的强度进行矩阵运算，重新计算的标记向量

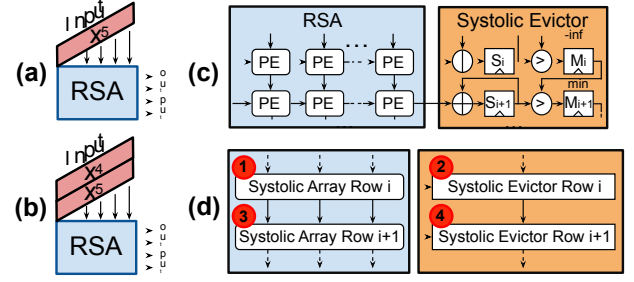


图 11: (a) 和 (b) 显示重新计算对 RSA 操作的影响。(c) RSA 和收缩期驱逐器的集成。(d) 脉动数组和脉动逐出器的执行顺序。红色圆圈中的数字表示订单。

可以与当前标记的输入向量组合以有效地创建输入矩阵。使用与图 6 (b) 中的示例相同的符号，图 11 (a) 显示当前令牌的输入向量 x_5 被发送到 RSA 进行 KV 向量计算。为了重新计算第四个令牌的 KV 向量，可以将 x_4 和 x_5 组合成一个矩阵，从而实现图 11(b) 中的最小延迟和能量增长。

5.3 收缩期驱逐器

AERP 算法中的令牌驱逐过程包括计算公式 1 中所述的注意力分数，根据公式 3 更新重要性分数，识别重要性分数最低的令牌，并执行 KV 缓存更新。

为了有效地实现驱逐算法，我们提出了一种收缩驱逐器 (SE)，它以收缩方式运行，并与 RSA 集成以动态搜索最小重要性分数。重要性得分是通过公式 1 中的 QK^T 结果求和而计算的，而不经 softmax 。这种集成确保一旦根据 RSA 计算出新代币的注意力分数，就能找到具有最低重要性分数的代币。在找到具有最小重要性分数的令牌的索引后，SE 将该索引发送到 eDRAM 控制器中的逐出控制器以逐出相应的令牌。图 11 (c) 说明了 SE 的设计及其在 RSA 中的集成。SE 包含一系列寄存器（在图 11 (c) 中表示为 S），用于预加载先前令牌的重要性分数，以及一个寄存器链（表示为 M），定期从上到下传播最小重要性分数 (min)。图 11 (d) 说明了 RSA 和 SE 的执行顺序。在一个周期内，从 RSA 的第 i 行计算注意力分数，然后 SE 的第 i 行更新重要性分数和最小重要性分数指数，分别标记为步骤 1 和步骤 2。在下一个周期中，在 RSA 和 SE 的下一行执行相同的操作，分别标记为步骤 3 和步骤 4。收缩驱逐器避免了最小搜索带来的额外 LLM 执行延迟。

6 凯勒调度程序

为了进一步最大限度地减少 eDRAM 刷新能量，我们引入了一种新颖的计算模式，可以缩短数据生命周期并加速 LLM 推理，同时不会影响准确性。

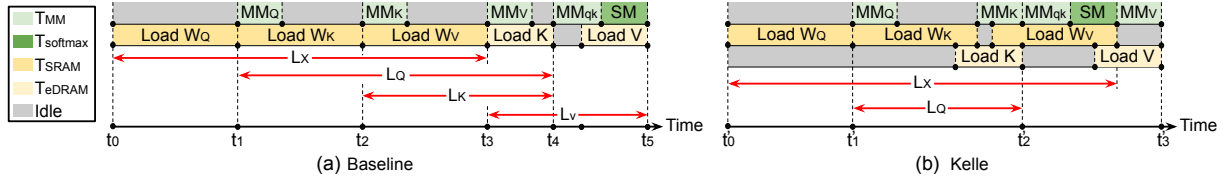


Figure 12: (a) and (b) show the computation patterns and eDRAM data lifetime of the SA block in baseline and Kelle scheduler. SM denotes softmax operation.

To begin, we perform a numerical analysis of the data lifetime associated with the self-attention (SA) architecture in the LLM decoding phase. As illustrated in Section 2.1, the computation in SA firstly involves matrix multiplication between the input X with weight matrices W_Q , W_K , and W_V producing the output Q , K and V . The processes are denoted as MM_Q , MM_K , and MM_V , respectively. Subsequently, Q and K undergo multiplication, followed by a softmax operation to compute the attention score A , which are referred to as MM_{qk} and SM , respectively. Ultimately, A is multiplied by the weight matrix W_O to produce the SA output, labeled as MM_O . The latency (T_{MM}) of matrix multiplications is estimated as follows:

$$T_{MM} = \frac{N_{MM}}{TOP_{RSA}} \quad (4)$$

where N_{MM} represents the number of MAC operations required by the matrix multiplication. TOP_{RSA} denotes the throughput of the RSA, as described in Section 5.2. The latency associated with eDRAM access operations for KV vectors, represented as T_{eDRAM} , is modeled as follows:

$$T_{eDRAM} = \frac{S_{KV}}{B_{eDRAM}} \quad (5)$$

where S_{KV} denotes the size of the KV vectors in bytes. B_{eDRAM} denotes the bandwidth of the eDRAM. Similarly, the latency associated with the weight SRAM access operations, denoted as T_{SRAM} , is modeled as follows:

$$T_{SRAM} = \frac{S_W}{B_{SRAM}} \quad (6)$$

where S_W denotes the size of weights in bytes. B_{SRAM} denotes the bandwidth of SRAM. Figures 12 (a) show a baseline computation pattern, where the matrix multiplication operations MM_Q , MM_K , MM_V , and MM_{qk} are conducted one after another, which prolongs the data lifetime for the inputs X , Q , K , and V . Data lifetime is defined as the interval from when data is computed to when it is utilized by a subsequent operation. For instance, in Figures 12 (a), the computation of vector Q starts at t_1 after the weight matrix W_Q is accessed from SRAM, and Q is consumed at t_4 when the multiplication of Q and K starts. Between t_1 and t_4 , W_K and W_V are loaded from SRAM and K is accessed from eDRAM KV cache. The latencies of accessing W_K and W_V are both T_{SRAM} and the latency of accessing K is T_{eDRAM} . So the data lifetime of Q is $2 \times T_{SRAM} + T_{eDRAM}$. The total data lifetime of all the activations is the sum of the data lifetime of each activation because they are all stored in eDRAM and require refreshing. We omit the computation time T_{MM} from Equation 4 due to its negligible magnitude relative to T_{SRAM} and T_{eDRAM} . This extended data lifetime leads to a higher refresh cost for eDRAM. The total data lifetime L_{bl} of the transient data in

baseline schedule is modeled as follows:

$$\begin{aligned} L_X &= 3 \times T_{SRAM}, L_Q = 2 \times T_{SRAM} + T_{eDRAM} \\ L_K &= T_{SRAM} + T_{eDRAM}, L_V = 2T_{eDRAM} \\ L_{bl} &= L_X + L_Q + L_K + L_V = 6T_{SRAM} + 4T_{eDRAM} \end{aligned} \quad (7)$$

where L_X , L_Q , L_K and L_V denotes the data lifetime of X , Q , K and V , respectively. T_{SRAM} and T_{eDRAM} are defined in Equation 6 and Equation 5. In contrast, the computation pattern used by the Kelle is illustrated in Figures 12 (b). Thanks to the integration of separate on-chip memory, the memory access for weights and KV vectors is parallelized. This arrangement reduces the data lifetime of activations, which can be estimated as follows:

$$\begin{aligned} L_X &= 3 \times T_{SRAM}, L_Q = T_{SRAM} + T_{eDRAM} \\ L_{Kelle} &= L_X + L_Q = 4T_{SRAM} + T_{eDRAM} \end{aligned} \quad (8)$$

The key and value vectors are used immediately for their respective computations, eliminating the need for long-term storage; therefore, their data lifetimes can be considered negligible. Compared to the baseline scheme, the Kelle scheduler significantly reduces the data lifetime of transient data in eDRAM, resulting in decreased refresh energy consumption and enhanced system performance.

7 Accuracy Evaluation

7.1 Main Accuracy Result

Kelle is evaluated on various LLMs, including Llama2 [75], Llama3 [21], Llama3.2 [21], Mistral [35], QWEN [87], and OPT [95] with varying model sizes. We evaluate Kelle on the language generation tasks by perplexity of WikiText-2 (WK2) [52], and PG19 [66]. WK2 sequences range from hundreds to thousands of tokens. PG19 sequences range from tens of thousands to millions. We evaluate the PG19 text generation task using the Cold Compress framework [4, 61] by providing the model a book title and a short description and setting the sequence generation length to 8192. Kelle is also evaluated over different zero-shot tasks, including PIQA (PQ) [10], Lambada (LA) [64], Arc Easy (A-e) [18], Arc Challenge (A-c) [18], TriviaQA (TQ) [36], and Qasper (QP) [19]. We use the LM Evaluation Harness [26] with default parameters.

For KV vector eviction, the number of tokens retained in the KV cache is dynamically adjusted based on the dataset during both pre-filling and decoding phases. To simulate bit flip error from low eDRAM refresh frequency, bit-level retention failure is introduced with a predefined probability based on the refresh interval. We set the refresh interval as 0.36ms, 5.4ms, 1.44ms, and 7.2ms for the MSBs (bits 15-8) of HST, LSBs (bits 7-0) of HST, MSBs of LST, LSBs of LST, respectively, with an average retention time of 1.05ms. This achieves an averaged retention failure rate at $2e-3$.

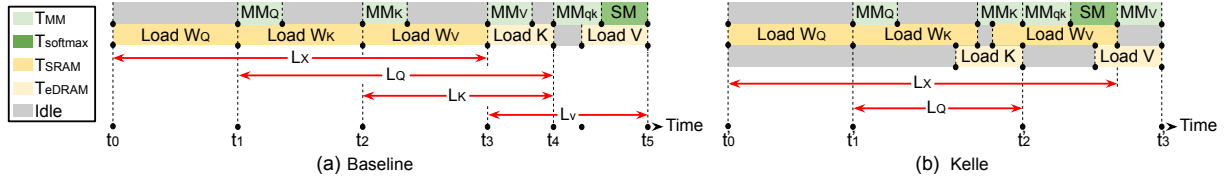


图 12: (a) 和 (b) 显示基线和 Kelle 调度程序中 SA 块的计算模式和 eDRAM 数据生命周期。SM 表示 softmax 运算。

首先，我们对 LLM 解码阶段与自注意力（SA）架构相关的数据生命周期进行数值分析。如第 2.1 节所示，SA 中的计算首先涉及输入 X 与权重矩阵 W_Q 、 W_K 和 W_V 之间的矩阵乘法，产生输出 Q 、 K 和 V 。这些进程分别表示为 MM_Q 、 MM_K 和 MM_V 。随后， Q 和 K 进行乘法，然后进行 softmax 运算来计算注意力分数 A ，分别称为 MM_{qk} 和 SM 。最终， A 乘以权重矩阵 W_O 以产生 SA 输出，标记为 MM_O 。矩阵乘法的延迟 (T_{MM}) 估计如下：

$$T_{MM} = \frac{N_{MM}}{TOP_{RSA}} \quad (4)$$

其中 N_{MM} 表示矩阵乘法所需的 MAC 运算次数。 TOP_{RSA} 表示 RSA 的吞吐量，如第 5.2 节所述。与 KV 向量的 eDRAM 访问操作相关的延迟（表示为 T_{eDRAM} ）建模如下：

$$T_{eDRAM} = \frac{S_{KV}}{B_{eDRAM}} \quad (5)$$

其中 S_{KV} 表示 KV 向量的大小（以字节为单位）。 B_{eDRAM} 表示 eDRAM 的带宽。类似地，与权重 SRAM 访问操作相关的延迟（表示为 T_{SRAM} ）建模如下：

$$T_{SRAM} = \frac{S_W}{B_{SRAM}} \quad (6)$$

其中 S_W 表示权重的大小（以字节为单位）。 B_{SRAM} 表示 SRAM 的带宽。图 12 (a) 显示了一种基线计算模式，其中矩阵乘法运算 MM_Q 、 MM_K 、 MM_V 和 MM_{qk} 依次进行，这延长了输入 X 、 Q 、 K 和 V 的数据生命周期。数据生命周期定义为从计算数据到后续操作使用数据的时间间隔。例如，在图 12 (a) 中，从 SRAM 访问权重矩阵 W_Q 后，向量 Q 的计算从 t_1 开始，而当 Q 和 K 相乘开始时， Q 在 t_4 处消耗。在 t_1 和 t_4 之间， W_K 和 W_V 从 SRAM 加载，而 K 从 eDRAM KV 缓存访问。访问 W_K 和 W_V 的延迟均为 T_{SRAM} ，访问 K 的延迟为 T_{eDRAM} 。因此 Q 的数据生命周期为 $2 \times T_{SRAM} + T_{eDRAM}$ 。所有激活的总数据生命周期是每次激活的数据生命周期之和，因为它们都存储在 eDRAM 中并且需要刷新。我们从公式 4 中省略了计算时间 T_{MM} ，因为其相对于 T_{SRAM} 和 T_{eDRAM} 的大小可以忽略不计。数据寿命的延长导致 eDRAM 的刷新成本更高。瞬态数据的总数据生命周期 L_{bl}

基线时间表建模如下：

$$\begin{aligned} L_X &= 3 \times T_{SRAM}, L_Q = 2 \times T_{SRAM} + T_{eDRAM} \\ L_K &= T_{SRAM} + T_{eDRAM}, L_V = 2T_{eDRAM} \\ L_{bl} &= L_X + L_Q + L_K + L_V = 6T_{SRAM} + 4T_{eDRAM} \end{aligned} \quad (7)$$

其中 L_X 、 L_Q 、 L_K 和 L_V 分别表示 X 、 Q 、 K 和 V 的数据生命周期。 T_{SRAM} 和 T_{eDRAM} 在等式 6 和等式 5 中定义。相比之下，Kelle 使用的计算模式如图 12 (b) 所示。由于集成了单独的片上存储器，权重和 KV 向量的存储器访问是并行的。这种安排减少了激活的数据寿命，可以按如下方式估计：

$$\begin{aligned} L_X &= 3 \times T_{SRAM}, L_Q = T_{SRAM} + T_{eDRAM} \\ L_{Kelle} &= L_X + L_Q = 4T_{SRAM} + T_{eDRAM} \end{aligned} \quad (8)$$

键和值向量立即用于各自的计算，无需长期存储；因此，它们的数据生命周期可以被认为可以忽略不计。与基准方案相比，Kelle 调度器显著缩短了 eDRAM 中瞬态数据的数据寿命，从而降低了刷新能耗并增强了系统性能。

7 准确度评估

7.1 主要精度结果

Kelle 在各种大语言模型进行了评估，包括 Llama2 [75]、Llama3 [21]、Llama3.2 [21]、Mistral [35]、QWEN [87] 和 OPT [95] 具有不同的模型尺寸。我们通过 WikiText-2 (WK2) [52] 和 PG19 [66] 的困惑度来评估 Kelle 在语言生成任务上的表现。WK2 序列的范围从数百到数千个标记。PG19 序列的范围从数万到数百万不等。我们使用 Cold Compress 框架 [4, 61] 评估 PG19 文本生成任务，为模型提供书名和简短描述，并将序列生成长度设置为 8192。Kelle 还针对不同的零样本任务进行评估，包括 PIQA (PIQ) [10]、Lambada (LA) [64]、Arc Easy (A-e) [18]、Arc Challenge (A-c) [18]、TriviaQA (TQ) [36] 和 Qasper (QP) [19]。我们使用具有默认参数的 LM 评估工具 [26]。

对于 KV 向量驱逐，KV 缓存中保留的令牌数量在预填充和解码阶段根据数据集动态调整。为了模拟低 eDRAM 刷新频率导致的位翻转错误，根据刷新间隔以预定义的概率引入位级保留故障。我们将 HST 的 MSB（位 15-8）、HST 的 LSB（位 7-0）、LST 的 MSB、LST 的 LSB 的刷新间隔分别设置为 0.36ms、5.4ms、1.44ms 和 7.2ms，平均保留时间为 1.05ms。这实现了 $2e-3$ 的平均保留失败率。

Table 2: Accuracy performance of each method. FP16 denotes the LLM accuracy under FP16 without KV cache reduction.

Model Method	LLaMA2-7B					LLaMA2-13B					LLaMA3.2-3B				LLaMA3-8B		Mistral-7B		QWEN2-7B			OPT-6.7B		
	FP16	SL	H2O	QR	Kelle	FP16	SL	H2O	QR	Kelle	FP16	SL	H2O	Kelle	FP16	Kelle	FP16	Kelle	FP16	H2O	Kelle	FP16	H2O	Kelle
WK2 (↓)	5.47	6.89	5.70	5.73	5.74	4.88	6.21	5.44	5.83	5.62	6.32	7.93	6.57	6.65	6.14	6.59	5.25	5.86	5.32	6.11	6.23	12.3	13.8	14.6
PG19 (↓)	10.51	NA	12.34	11.77	12.59	8.75	NA	9.84	9.15	9.80	8.95	NA	9.84	9.66	11.82	12.97	10.42	13.30	9.3	11.2	11.4	17.4	20.1	19.8
A-c (↑)	46.33	38.40	46.10	45.80	45.93	49.06	46.08	48.35	47.59	48.83	50.34	46.59	50.08	50.03	53.16	51.89	55.03	54.98	57.6	56.1	55.8	45.2	44.1	44.8
A-e (↑)	74.62	52.99	73.02	72.89	72.78	77.48	58.25	76.71	76.45	76.33	76.98	70.45	76.31	76.45	77.69	75.81	80.22	78.90	70.5	67.8	68.2	60.1	58.8	58.6
PQ (↑)	79.11	75.14	78.05	77.42	77.35	80.52	78.92	79.05	78.78	78.92	79.7	75.78	79.43	78.64	80.52	77.62	82.15	80.81	80.9	78.4	78.5	76.4	75.2	74.9
LA (↑)	73.90	NA	NA	72.39	72.81	76.75	NA	NA	75.67	75.98	73.58	NA	NA	71.26	75.72	73.38	75.49	75.54	68.4	NA	67.1	61.5	NA	59.2
TQ (↑)	48.95	NA	47.53	47.56	47.40	58.73	NA	57.46	56.86	57.55	59.84	NA	58.62	58.70	61.53	58.79	61.57	59.96	63.5	61.1	60.7	43.2	40.3	40.6
QP (↑)	12.69	NA	12.31	12.06	12.18	13.07	NA	11.75	11.63	11.86	12.52	NA	10.87	11.01	13.84	11.88	13.18	11.67	21.3	19.4	19.5	9.3	7.8	7.5

Table 3: LLaMA2-7B accuracies over different cache sizes.

N'	Full	512	256	128	64	32	16
A-c (↑)	46.02	46.02	45.92	45.93	44.63	44.20	38.52
A-e (↑)	73.05	73.04	73.05	72.78	72.38	70.42	67.27
PQ (↑)	78.75	78.49	77.61	77.35	75.31	74.14	71.63

Table 4: LLaMA2-7B accuracies of different refresh intervals.

Uniform (μ s)	540		1050		2062	
HST (μ s)	180, 3600		360, 5400		720, 9000	
LST (μ s)	720, 5400		1440, 7200		2880, 10800	
Accuracy	Uniform	2DRP	Uniform	2DRP	Uniform	2DRP
A-c (↑)	45.31	46.26	44.19	45.93	38.52	39.78
A-e (↑)	72.85	73.12	70.29	72.78	66.50	67.05
PQ (↑)	76.83	77.44	76.43	77.35	74.97	75.21

We compare the accuracy performance of the Kelle algorithm with a state-of-the-art quantization framework, QuaRot (QR) [6]. Additionally, we include StreamLLM [83] and H2O [98], two recent KV cache eviction techniques, for comparison. The model weights are quantized with 8-bit across all the approaches. To keep the same KV cache budget between quantization and KV cache eviction baselines, we configure QuaRot to quantize the KV vectors to 4-bit, while StreamLLM, H2O, and Kelle are left unquantized as 16-bit. For Kelle, we maintain a token storage budget of $N'=128$ for the PQ, LA, A-e and A-c, $N'=512$ for WK2, $N'=1024$ for the TQ and QP, and $N'=2048$ for PG19. Within the token budget, the most recent token window size is configured as 64 for PQ, LA, A-e, and A-c; 256 for WK2; 512 for TQ and QP; and 1024 for PG-19. 10 initial tokens are also preserved across the datasets. StreamLLM and H2O are set to have the same token storage budget as Kelle. We also compare with the original FP16 models without KV cache eviction, denoted as FP16. As shown in Table 2, Kelle maintains a comparable accuracies as the original full KV cache model and outperforms or achieves a comparable performance as the rest methods, showing the superior accuracy performance of AERP and 2DRP algorithms.

7.2 Ablation Study

We adjust the budget size N' for the Llama2-7B model and examine its impact across various tasks. All other settings (e.g., quantization bitwidth, retention failure rate) for Kelle remain the same. From Table 3, we observe a consistent decline in accuracy as the budget N' decreases, but still achieves reasonable performance for $N' \geq 128$, comparing with the KV cache without pruning (Full in Table 3).

Next, we examine the impact of 2DRP on LLM accuracy. Specifically, we compare 2DRP with a condition where all eDRAM cells

Table 5: Kelle Qualitative Metrics

Model	Method	CNN	Truth	BBQ
LLaMA2-7B	FP16	40.58	34.28	95.21
	Kelle	38.54	33.26	93.75
Mistral-7B	FP16	36.13	36.31	96.11
	Kelle	34.61	34.89	94.83

Table 6: Accuracy of Kelle with Quantization

Method	Kelle W8A16	Kelle W4A8
WK2 (↓)	5.74	6.51
AC (↑)	45.93	44.89
AE (↑)	72.78	69.96
PQ (↑)	77.35	76.70

share the same refresh interval, while maintaining the same average retention failure rate as 2DRP. All other conditions remain the same for Kelle. In Table 4, we present the accuracy change with varying refresh intervals on different tasks for the Llama2-7B model. In Table 4, Uniform (μ s) denotes the uniform refresh interval applied to eDRAM. The two numbers in the HST row represent the refresh intervals for the MSBs and LSBs of HSTs, same for the LST row. We observe that 2DRP improves accuracy compared to the uniform eDRAM refresh across all the conditions and datasets.

Since edge deployment often involves human-facing applications, it is important to evaluate the impact of the approximations in memory behavior introduced by 2DRP on text generation qualitative metrics. To evaluate coherence, we run Kelle on the LLaMA2-7B and Mistral-7B models using the CNN/DailyMail [53] (CNN) summarization dataset and report the ROUGE-1 scores. To evaluate factual correctness, we test Kelle on the TruthfulQA benchmark [43] (Truth) and report the multiple-choice, single-answer accuracy. To assess bias tendencies, we use the BBQ benchmark [58] and report the corresponding bias evaluation scores for both models. The results in Table 5 show that Kelle achieves performance comparable to the FP16 model across all criteria.

Finally, we quantize the Llama2-7B model using the Quarot framework [6] which adopts Hadamard Transformation to enable low bit LLM quantization. We quantize the model weights to 4-bit, KV vectors, and activations to 8-bit. With quantization, Kelle's system performance is expected to improve further, while the impact on accuracy remains minimal, as shown in Table 6. This demonstrates Kelle's compatibility with model quantization techniques.

8 Hardware Evaluation

In this section, we report the hardware evaluation results of Kelle edge accelerator described in Section 5. The Kelle edge accelerator consists of a 2D 32×32 RSA, an SFU, essential interfaces, and a memory controller, all implemented in RTL using SystemVerilog, with frequency set to 1GHz. We report the area and power of Kelle accelerator by synthesizing the components using 45nm NanGate Open Cell Library [1] with Synopsys Design Compiler [9]. The size of SRAM for weight storage is set to 2MB. The size of eDRAM for KV

表 2: 每种方法的准确度性能。FP16表示没有KV缓存减少的情况下FP16下的LLM精度

Model Method	LLaMA2-7B					LLaMA2-13B					LLaMA3.2-3B				LLaMA3-8B		Mistral-7B		QWEN2-7B			OPT-6.7B		
	FP16	SL	H2O	QR	Kelle	FP16	SL	H2O	QR	Kelle	FP16	SL	H2O	Kelle	FP16	Kelle	FP16	Kelle	FP16	H2O	Kelle	FP16	H2O	Kelle
WK2 (↓)	5.47	6.89	5.70	5.73	5.74	4.88	6.21	5.44	5.83	5.62	6.32	7.93	6.57	6.65	6.14	6.59	5.25	5.86	5.32	6.11	6.23	12.3	13.8	14.6
PG19 (↓)	10.51	NA	12.34	11.77	12.59	8.75	NA	9.84	9.15	9.80	8.95	NA	9.84	9.66	11.82	12.97	10.42	13.30	9.3	11.2	11.4	17.4	20.1	19.8
A-c (↑)	46.33	38.40	46.10	45.80	45.93	49.06	46.08	48.35	47.59	48.83	50.34	46.59	50.08	50.03	53.16	51.89	55.03	54.98	57.6	56.1	55.8	45.2	44.1	44.8
A-e (↑)	74.62	52.99	73.02	72.89	72.78	77.48	58.25	76.71	76.45	76.33	76.98	70.45	76.31	76.45	77.69	75.81	80.22	78.90	70.5	67.8	68.2	60.1	58.8	58.6
PQ (↑)	79.11	75.14	78.05	77.42	77.35	80.52	78.92	79.05	78.78	78.92	79.7	75.78	79.43	78.64	80.52	77.62	82.15	80.81	80.9	78.4	78.5	76.4	75.2	74.9
LA (↑)	73.90	NA	NA	72.39	72.81	76.75	NA	NA	75.67	75.98	73.58	NA	NA	71.26	75.72	73.38	75.49	75.54	68.4	NA	67.1	61.5	NA	59.2
TQ (↑)	48.95	NA	47.53	47.56	47.40	58.73	NA	57.46	56.86	57.55	59.84	NA	58.62	58.70	61.53	58.79	61.57	59.96	63.5	61.1	60.7	43.2	40.3	40.6
QP (↑)	12.69	NA	12.31	12.06	12.18	13.07	NA	11.75	11.63	11.86	12.52	NA	10.87	11.01	13.84	11.88	13.18	11.67	21.3	19.4	19.5	9.3	7.8	7.5

表 3: 不同缓存大小下的 LLaMA2-7B 精度。

N'	Full	512	256	128	64	32	16
A-c (↑)	46.02	46.02	45.92	45.93	44.63	44.20	38.52
A-e (↑)	73.05	73.04	73.05	72.78	72.38	70.42	67.27
PQ (↑)	78.75	78.49	77.61	77.35	75.31	74.14	71.63

表 4: 不同刷新间隔的 LLaMA2-7B 精度。

Uniform (μ s)	540		1050		2062	
HST (μ s)	180, 3600		360, 5400		720, 9000	
LST (μ s)	720, 5400		1440, 7200		2880, 10800	
Accuracy	Uniform	2DRP	Uniform	2DRP	Uniform	2DRP
A-c (↑)	45.31	46.26	44.19	45.93	38.52	39.78
A-e (↑)	72.85	73.12	70.29	72.78	66.50	67.05
PQ (↑)	76.83	77.44	76.43	77.35	74.97	75.21

我们将 Kelle 算法的准确性性能与最先进的量化框架 QuaRot (QR) [6] 进行比较。此外，我们还包括 StreamLLM [83] 和 H2O [98] 这两种最新的 KV 缓存驱逐技术，以进行比较。所有方法的模型权重均采用 8 位量化。为了在量化和 KV 缓存驱逐基线之间保持相同的 KV 缓存预算，我们将 QuaRot 配置为将 KV 向量量化为 4 位，而 StreamLLM、H2O 和 Kelle 则未量化为 16 位。对于 Kelle，我们为 PQ、LA、A-e 和 A-c 维持代币存储预算为 $N'=128$ ，为 WK2 维持代币存储预算为 $N'=512$ ，为 TQ 和 QP 维持代币存储预算为 $N'=1024$ ，为 PG19 维持代币存储预算为 $N'=2048$ 。在代币预算内，PQ、LA、A-e、A-c 的最新代币窗口大小配置为 64；WK2 为 256；512 用于 TQ 和 QP；PG-19 为 1024。数据集中还保留了 10 个初始标记。StreamLLM 和 H2O 设置为与 Kelle 具有相同的代币存储预算。我们还与没有 KV 缓存驱逐的原始 FP16 模型进行比较，记为 FP16。如表 2 所示，Kelle 保持了与原始全 KV 缓存模型相当的精度，并且优于或达到了与其他方法相当的性能，显示了 AERP 和 2DRP 算法优越的精度性能。

7.2 消融研究

我们调整 Llama2-7B 模型的预算大小 N' 并检查其对各种任务的影响。Kelle 的所有其他设置（例如量化位宽、保留失败率）保持不变。从表 3 中，我们观察到随着预算 N' 的减少，准确度持续下降，但与没有修剪的 KV 缓存相比（表 3 中的完整内容）， $N' \geq 128$ 仍然实现了合理的性能。

接下来，我们研究 2DRP 对 LLM 准确性的影响。具体来说，我们将 2DRP 与所有 eDRAM 单元的条件进行比较

表 5: Kelle 定性指标

Model	Method	CNN	Truth	BBQ
LLaMA2-7B	FP16	40.58	34.28	95.21
	Kelle	38.54	33.26	93.75
Mistral-7B	FP16	36.13	36.31	96.11
	Kelle	34.61	34.89	94.83

表 6: Kelle 量化精度

Method	Kelle W8A16	Kelle W4A8
WK2 (↓)	5.74	6.51
AC (↑)	45.93	44.89
AE (↑)	72.78	69.96
PQ (↑)	77.35	76.70

共享相同的刷新间隔，同时保持与 2DRP 相同的平均保留故障率。对于凯勒来说，所有其他条件保持不变。在表 4 中，我们展示了 Llama2-7B 模型在不同任务上随刷新间隔变化的精度变化。在表 4 中，Uniform (μ s) 表示应用于 eDRAM 的统一刷新间隔。HST 行中的两个数字代表 HST 的 MSB 和 LSB 的刷新间隔，LST 行也是如此。我们观察到，与所有条件和数据集上的统一 eDRAM 刷新相比，2DRP 提高了准确性。

由于边缘部署通常涉及面向人类的应用程序，因此评估 2DRP 引入的内存行为近似值对文本生成定性指标的影响非常重要。为了评估一致性，我们使用 CNN/DailyMail [53] (CNN) 总结数据集在 LLaMA2-7B 和 Mistral-7B 模型上运行 Kelle 并报告 ROUGE-1 分数。为了评估事实的正确性，我们在 TruthfulQA 基准 [43] (Truth) 上测试 Kelle 并报告多项选择、单答案的准确性。为了评估偏差倾向，我们使用 BBQ 基准 [58] 并报告两个模型相应的偏差评估分数。表 5 中的结果表明，Kelle 在所有标准上都达到了与 FP16 模型相当的性能。

最后，我们使用 Quarot 框架 [6] 量化 Llama2-7B 模型，该框架采用 Hadamard 变换来实现低位 LLM 量化。我们将模型权重量化为 4 位、KV 向量，并将激活量化为 8 位。通过量化，Kelle 的系统性能预计将进一步提高，而对精度的影响仍然很小，如表 6 所示。这证明了 Kelle 与模型量化技术的兼容性。

8 硬件评估

在本节中，我们报告第 5 节中描述的 Kelle 边缘加速器的硬件评估结果。Kelle 边缘加速器由 2D 32×32 RSA、SFU、基本接口和内存控制器组成，全部使用 SystemVerilog 在 RTL 中实现，频率设置为 1GHz。我们通过使用 45nm NanGate Open Cell Library [1] 和 Synopsys Design Compiler [9] 合成组件来报告 Kelle 加速器的面积和功率。用于权值存储的 SRAM 大小设置为 2 MB。KV 的 eDRAM 大小

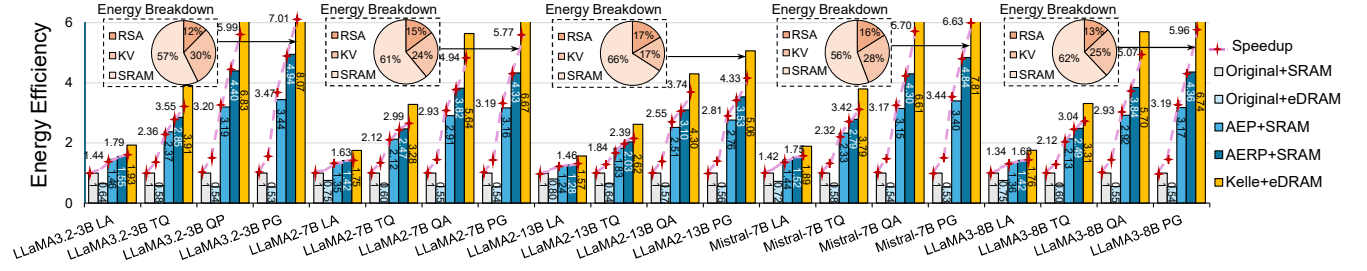


Figure 13: Comparison between Kelle and baseline systems. The performance are evaluated in terms of normalized energy efficiency and speedup. The pie charts show the on-chip energy breakdown of major components within Kelle+eDRAM. The dotted red lines depict the speedup of the corresponding settings.

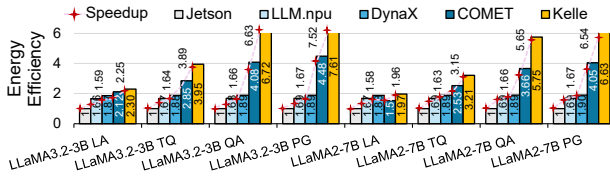


Figure 14: Comparison between LLM accelerators

cache and activation storage is set to 4MB and 256KB respectively. SRAM and eDRAM bandwidths are set to 128GB/s and 256 GB/s, respectively. We utilize Destiny [60] to evaluate the area, power, and timing performance of the eDRAM and SRAM with 65nm tech node at 105°C. The eDRAM retention time distribution aligns with the data shown in Figure 4 [38, 97]. Notably, eDRAM operating at temperatures below 105°C exhibits an even longer retention time, further enhancing system performance. We utilize Cacti 7 [8] to simulate the performance of a 16GB LPDDR4 DRAM, with 64GB/s bandwidth, similar to the DRAM in the Google Coral edge device [72]. With these settings, the total on-chip area is 9.5mm² and the area breakdown of RSA, eDRAM, SRAM, SFU are 23%, 33%, 37%, 7%, respectively. The DRAM takes an area of 16mm². The on-chip power is 6.52W and the power breakdown of RSA, eDRAM, SRAM, SFU are 17%, 29%, 41%, 13%, respectively. The DRAM power is 11.74W. Kelle accelerator achieves 4.13 INT8 TOPs. The Kelle scheduler described in Section 6 further reduces the eDRAM cost.

We assess the hardware performance of the Kelle accelerator across various LLM architectures over multiples tasks including Lambada (LA) [64], TriviaQA (TQ) [36], Qasper (QA) [19], and PG19 [66], with the context length set to 128, 512, 1024, and 512, and the decoding length set to 512, 2048, 5120, and 8192, respectively. The batch size is set to 16. The off-chip DRAM access latency and energy are included in all evaluation results.

8.1 End-to-End Performance Evaluation

8.1.1 Evaluation Baseline. To understand the separate contributions of the Kelle algorithm and eDRAM-based accelerator discussed in Section 5, we compare the Kelle algorithm paired with an eDRAM-based Kelle accelerator, referred to as **Kelle+eDRAM**, against four baseline solutions.

The first baseline, **Original+SRAM**, runs the original LLM on a system using SRAM as the primary on-chip storage. The model

weights are quantized to 8 bits, activations and KV vectors remain 16 bits and processed using Kelle RSA configured for 8-bit MAC operations. The KV cache remains intact, with no AERP applied. The SRAM-based system is configured to match the total on-chip area of Kelle+eDRAM. We adjust the SRAM and systolic size to achieve balanced compute/memory IO ratio, resulting in a systolic array with 24×24 8-bit PEs, 4MB of on-chip SRAM, 16GB of off-chip DRAM. The second baseline, **Original+eDRAM**, involves running the original LLM on an eDRAM-based Kelle accelerator while keeping the KV cache intact. The models are processed using Kelle RSA configured for 8-bit MAC operations. This baseline removes all algorithmic innovations and evaluates only the performance of the eDRAM-based system. In the third baseline, **AEP+SRAM**, we apply the attention-based eviction techniques with the settings described in Section 7.1 for KV cache pruning, implemented on the same SRAM-based system of Original+SRAM. The goal is to evaluate the impact of the cache eviction algorithm on the SRAM-based system. Note that this baseline does not involve any recomputation. The fourth baseline, **AERP+SRAM**, runs the AERP algorithm on the SRAM-based Kelle accelerator.

8.1.2 End-to-End Performance Improvement. Figure 13 compares the above baseline solutions in terms of energy efficiency and processing latency on multiple LLM and datasets. On average, Kelle+eDRAM achieves a 3.94× and 4.46× improvement in latency and energy efficiency compared to the Original+SRAM, and performance gap gets larger as the decoding sequence gets longer. The superior performance of Kelle stems from the algorithmic innovations of AERP and 2DRP, combined with hardware advantages such as the efficient eDRAM memory controller, the systolic evictor design, and the proposed Kelle scheduler.

8.1.3 Individual Contribution on the Performance Improvement. In this section, we investigate the individual impact of Kelle optimization techniques. First, on average, compared to Original+SRAM, Original+eDRAM improves the speedup by 32% but degrades the energy efficiency by 39%. The increased energy consumption is attributed to eDRAM refresh operations without algorithmic or hardware-level optimizations. The eDRAM enhances speedup due to its larger capacity and faster access speeds compared to SRAM. Second, the attention-based eviction policy accelerated by the systolic evictor reduces the latency by 2.39× and improves the energy efficiency by 2.41×, when comparing the AEP+SRAM system with

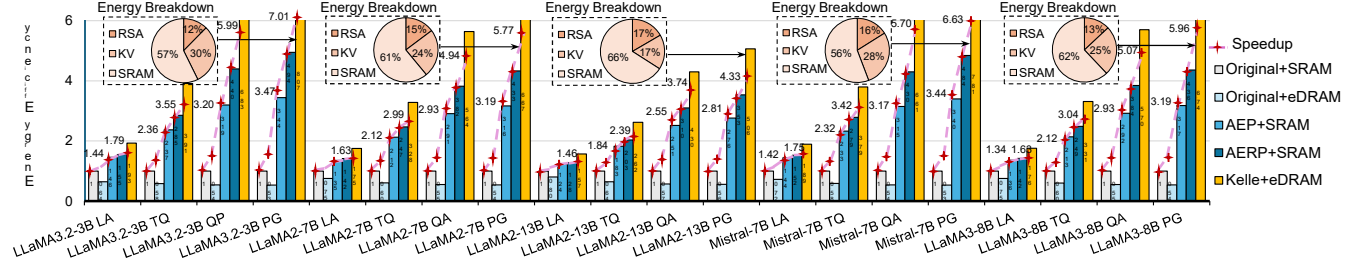


图 13: Kelle 和基线系统之间的比较。性能是根据标准化能源效率和加速来评估的。饼图显示了 Kelle+eDRAM 内主要组件的片上能耗细分。红色虚线表示相应设置的加速比。

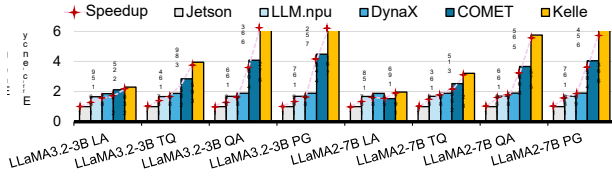


图 14: LLM 加速器之间的比较

缓存和激活存储分别设置为4MB和256KB。SRAM 和 eDRAM 带宽分别设置为 128GB/s 和 256GB/s。我们利用 Destiny [60] 评估 105°C 下 65nm 技术节点的 eDRAM 和 SRAM 的面积、功耗和时序性能。eDRAM 保留时间分布与图 4 [38, 97] 中显示的数据一致。值得注意的是, eDRAM 在低于 105°C 的温度下运行时表现出更长的保留时间, 进一步增强了系统性能。我们利用 Cacti 7 [8] 来模拟 16GB LPDDR4 DRAM 的性能, 带宽为 64GB/s, 类似于 Google Coral 边缘设备中的 DRAM [72]。通过这些设置, 片上总面积为 9.5mm², RSA、eDRAM、SRAM、SFU 的面积细分分别为 23%、33%、37%、7%。DRAM 占用的面积为 16mm²。片上功耗为 6.52W, RSA、eDRAM、SRAM、SFU 的功耗分别为 17%、29%、41%、13%。DRAM 功率为 11.74W。Kelle 加速器达到 4.13 INT8 TOP。第 6 节中描述的 Kelle 调度程序进一步降低了 eDRAM 成本。

我们评估了 Kelle 加速器在各种 LLM 架构上在多个任务上的硬件性能, 包括 Lambada (LA) [64]、TriviaQA (TQ) [36]、Qasper (QA) [19] 和 PG19 [66], 上下文长度设置为 128、512、1024 和 512, 解码长度设置为 512、2048、5120 和分别为 8192。批量大小设置为 16。所有评估结果均包含片外 DRAM 访问延迟和能量。

8.1 端到端性能评估

8.1.1 评估基线。为了了解第 5 节中讨论的 Kelle 算法和基于 eDRAM 的加速器的各自贡献, 我们将 Kelle 算法与基于 eDRAM 的 Kelle 加速器 (称为 Kelle+eDRAM) 配对与四种基准解决方案进行比较。

第一个基线, Original+SRAM, 在使用 SRAM 作为主要片上存储的系统上运行原始 LLM。型号

权重被量化为 8 位, 激活和 KV 向量保持为 16 位, 并使用为 8 位 MAC 操作配置的 Kelle RSA 进行处理。KV 缓存保持不变, 未应用 AERP。基于 SRAM 的系统配置为与 Kelle+eDRAM 的总片上面积相匹配。我们调整 SRAM 和脉动大小以实现平衡的计算/内存 IO 比率, 从而形成具有 24×24 个 8 位 PE、4 MB 片上 SRAM、16GB 片外 DRAM 的脉动阵列。第二个基线, Original+eDRAM, 涉及在基于 eDRAM 的 Kelle 加速器上运行原始 LLM, 同时保持 KV 缓存完好无损。这些模型使用配置为 8 位 MAC 操作的 Kelle RSA 进行处理。该基准消除了所有算法创新, 仅评估基于 eDRAM 的系统性能。在第三个基线 AEP+SRAM 中, 我们应用基于注意力的驱逐技术以及第 7.1 节中描述的 KV 缓存修剪设置, 在与 Original+SRAM 相同的基于 SRAM 的系统上实现。目标是评估缓存逐出算法对基于 SRAM 的系统的影响。请注意, 此基线不涉及任何重新计算。第四条基线, AERP+SRAM, 在基于 SRAM 的 Kelle 加速器上运行 AERP 算法。

8.1.2 端到端性能改进。图 13 对多个 LLM 和数据集的能源效率和处理延迟方面的上述基准解决方案进行了比较。平均而言, 与原始+SRAM 相比, Kelle+eDRAM 在延迟和能源效率方面实现了 3.94× 和 4.46× 的改进, 并且随着解码序列变长, 性能差距变得更大。Kelle 的优越性能源于 AERP 和 2DRP 的算法创新, 结合高效的 eDRAM 内存控制器、脉动驱逐器设计和提出的 Kelle 调度器等硬件优势。

8.1.3 个人对绩效改进的贡献。在本节中, 我们研究 Kelle 优化技术的个体影响。首先, 平均而言, 与 Original+SRAM 相比, Original+eDRAM 加速比提高了 32%, 但能源效率降低了 39%。能耗增加归因于没有算法或硬件级优化的 eDRAM 刷新操作。与 SRAM 相比, eDRAM 具有更大的容量和更快的访问速度, 从而提高了加速速度。其次, 与 AEP+SRAM 系统和

Table 7: Energy efficiency over multiple KV cache budgets.

N' in PG19	2048	3500	5250	7000	8750
LLaMA3.2-3B	8.07×	6.89×	5.77×	5.13×	4.55×
LLaMA2-13B	5.06×	4.62×	4.02×	3.46×	3.11×

the Original+SRAM system. Next, thanks to the attention-based recomputation policy, AERP+SRAM system improves the speedup and energy efficiency by 1.19× and 1.27× to the AEP+SRAM system. Finally, for the system executing models with AERP, the eDRAM optimized by 2DRP and Kelle scheduler provides a 1.29× improvement in speedup and a 1.45× improvement in energy efficiency when comparing Kelle+eDRAM to AERP+SRAM system. Specifically, the 2DRP mechanism greatly reduces refresh energy, enabling the Kelle to take full advantage of eDRAM.

8.1.4 Overhead Analysis. The pie charts in Figure 13 display the energy breakdown for the Kelle+eDRAM system. The reduced energy share for the KV cache highlights how eDRAM coupled with Kelle algorithms alleviates memory access bottlenecks. Thanks to the efficiency of systolic arrays for matrix-matrix multiplication, the hardware overhead from KV recomputation is minimal, with RSA consuming only a small portion of on-chip energy.

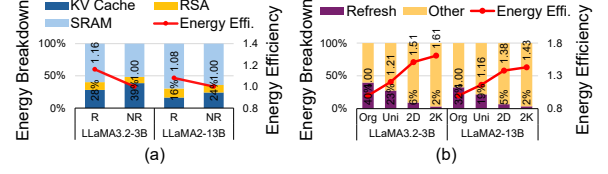
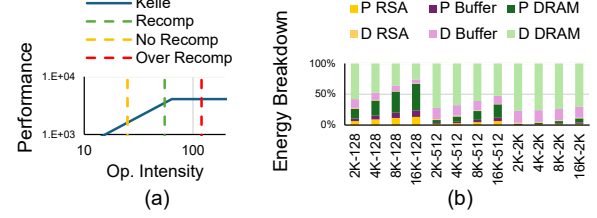
To accelerate the token eviction process, we introduce a Systolic Evictor unit, which is a small computation unit coupled with the RSA. It takes an area of $0.06mm^2$ (0.6% of the on-chip area) and consumes power of 0.028W (0.4% of the on-chip power). The systolic evictor avoids the stall of the LLM execution for KV cache eviction and redundant memory and computation access. It improves the system's energy efficiency and reduces the latency by 5% and 7%.

8.2 Comparison with Other Accelerators

We compare Kelle+eDRAM with other cutting-edge LLM accelerators. LLM.npu [86] enhances the on-device Neural Processing Unit (NPU) offloading to reduce pre-filling latency by re-constructing the prompt and model. DynaX [85] proposes dynamic fine-grained structured pruning to enhance the efficiency of sparse attention computation and achieves a 90% attention sparsity. Dynax alleviates the computation bottleneck during the pre-filling stage. COMET [46] quantizes the LLMs to 4-bit and designs high performance GPU kernel to support the mixed-precision computation. As advanced quantization techniques are not the main focus of this paper, we configure COMET to quantize LLM weights to 8-bit and both activations and KV vectors to 4-bit, ensuring a comparable KV cache storage budget to that of Kelle+eDRAM. Finally, we compare Kelle with the NVIDIA Jetson Orin edge GPU [2] implementation of LLM using FP8, measured with pynvml [3] and nvidia-smi [55].

As shown in Figure 14, Kelle+eDRAM achieves better improvements in speedup and energy efficiency over other LLM accelerators. LLM.npu and Dynax optimize the computation-intensive pre-filling stage but do not address the KV cache bottleneck encountered during the LLM decoding stage. The performance gains of Kelle over COMET underscore the limitations of relying solely on KV cache compression without dedicated hardware accelerator support.

8.3 Ablation Study

**Figure 15: (a) Impact of KV cache recomputation in Kelle+eDRAM. (b) Evaluation on 2DRP and Kelle scheduler.****Figure 16: (a) KV Cache Recomputation impact. (b) Evaluation on long input sequences. P and D denote the prefilling and decoding stage, respectively.**

8.3.1 Impact of KV Cache Budget. Table 7 illustrates the energy efficiency improvement of Kelle+eDRAM over different KV cache budgets N' . Without eviction, the largest possible number of tokens will be $N' = 8750$ for PG19. Results indicate that even under this condition, Kelle achieves approximately 3× greater energy efficiency over Original+SRAM, highlighting the robustness of Kelle.

8.3.2 Impact of the Recomputation. We compare the energy consumption of Kelle+eDRAM with and without KV cache recomputation. As shown in Figure 15 (a), the recomputation algorithm effectively reduces the energy consumption of KV cache, with minimal increase in RSA energy consumption. Moreover, we profile the popularity change of tokens during the pre-filling and decoding stage across LLM architectures and tasks. On average, over 86% of the popular tokens in the pre-filling stage continue to be popular in the decoding stage, validating the execution strategy outlined in Section 4.1.2.

Recomputation allows Kelle to store more tokens on-chip, reducing DRAM access. When processing the LLaMA2-7B model, accessing one KV vector from DRAM takes approximately 1.1 μs . In comparison, recomputing a KV vector using the RSA introduces an additional latency of 3.2 μs . Recomputation helps hide memory stalls by overlapping compute with memory access, reducing overall latency and improving energy efficiency by an average of 25%. For example, loading four KV vectors from DRAM requires 4.4 μs . With recomputation, three vectors are loaded, and one is recomputed in parallel during the load, reducing the total latency to 3.3 μs . In terms of energy, the RSA remains active regardless of the number of input vectors, so the incremental energy cost of recomputation is negligible.

Figure 16 (a) presents the roofline model of Kelle under three settings: No Recomp (without recomputation), Recomp (with a moderate recomputation workload), and Over Recomp (with excessive

表 7: 多个 KV 缓存预算的能源效率。

N' in PG19	2048	3500	5250	7000	8750
LLaMA3.2-3B	8.07×	6.89×	5.77×	5.13×	4.55×
LLaMA2-13B	5.06×	4.62×	4.02×	3.46×	3.11×

原始的+SRAM 系统。接下来，得益于基于注意力的重新计算策略，AERP+SRAM系统将AEP+SRAM系统的加速比和能源效率提高了1.19×和1.27×。最后，对于使用 AERP 执行模型的系统，与 Kelle+eDRAM 与 AERP+SRAM 系统相比，通过 2D RP 和 Kelle 调度程序优化的 eDRAM 提供了 1.29× 的加速改进和 1.45× 的能源效率改进。具体来说，2DRP机制大大降低了刷新能耗，使Kelle能够充分利用eDRAM。

8.1.4 开销分析。图 13 中的饼图显示了 Kelle+eDRAM 系统的能源细分。KV 缓存能量份额的减少凸显了 eDRAM 与 Kelle 算法相结合如何缓解内存访问瓶颈。由于脉动阵列在矩阵-矩阵乘法方面的效率很高，KV 重新计算的硬件开销很小，RSA 仅消耗一小部分片上能量。

为了加速令牌驱逐过程，我们引入了 Systolic Evictor 单元，它是与 RSA 耦合的小型计算单元。它占用的面积为片上面积)的 $0.06mm^2$ (0.6%，功耗为 0.028W (片上功率的 0.4%)。脉动逐出器避免了 KV 缓存逐出以及冗余内存和计算访问的 LLM 执行停顿。它提高了系统的能源效率，并将延迟降低了 5% 和 7%。

8.2 与其他加速器的比较

我们将 Kelle+eDRAM 与其他尖端 LLM 加速器进行比较。LLM.npu [86] 增强了设备上的神经处理单元 (NPU) 卸载，通过重新构建提示和模型来减少预填充延迟。DynaX [85] 提出动态细粒度结构化剪枝来提高稀疏注意力计算的效率，并实现了 90% 的注意力稀疏度。DynaX 缓解了预填充阶段的计算瓶颈。COMET [46] 将 LLM 量化为 4 位，并设计高性能 GPU 内核来支持混合精度计算。由于高级量化技术不是本文的主要焦点，因此我们将 COMET 配置为将 LLM 权重量化为 8 位，将激活和 KV 向量量化为 4 位，确保 KV 缓存存储预算与 Kelle+eDRAM 相当。最后，我们将 Kelle 与使用 FP8 的 LLM 的 NVIDIA Jetson Orin 边缘 GPU [2] 实现进行比较，并使用 pynvml [3] 和 nvidia-smi [55] 进行测量。

如图 14 所示，Kelle+eDRAM 比其他 LLM 加速器在加速和能源效率方面取得了更好的改进。LLM.npu和DynaX优化了计算密集型预填充阶段，但没有解决LLM解码阶段遇到的KV缓存瓶颈。Kelle 相对于 COMET 的性能提升凸显了仅依赖 KV 缓存压缩而没有专用硬件加速器支持的限制性。

8.3 Ablation Study

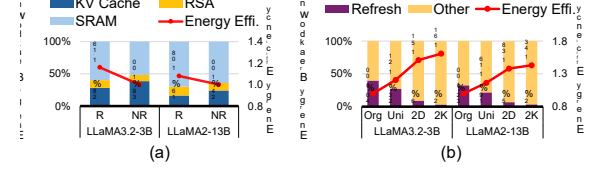


图 15: (a) KV 缓存重新计算对 Kelle+eDRAM 的影响。(b) 对 2DRP 和 Kelle 调度器的评估。

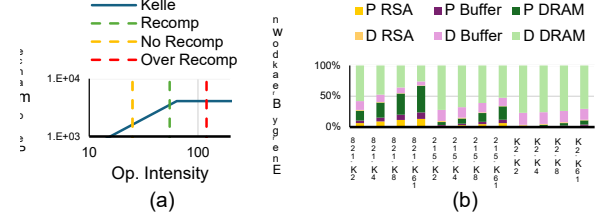


图 16: (a) KV 缓存重新计算影响。(b) 对长输入序列的评估。P和D分别表示预填充和解码阶段。

8.3.1 KV 缓存预算的影响。表 7 说明了 Kelle+eDRAM 相对于不同 KV 缓存预算 N' 的能效改进。如果没有驱逐，PG19 的最大可能代币数量将为 $N' = 8750$ 。结果表明，即使在这种情况下，Kelle 也比原始 SRAM 实现了约 3× 高的能源效率，凸显了 Kelle 的稳健性。

8.3.2 重新计算的影响。我们比较了有和没有 KV 缓存重新计算的 Kelle+eDRAM 的能耗。如图 15 (a) 所示，重计算算法有效降低了 KV 缓存的能耗，并且 RSA 能耗的增加最小。此外，我们还分析了 LLM 架构和任务中预填充和解码阶段代币流行度的变化。平均而言，超过 86% 的预填充阶段流行代币在解码阶段继续流行，验证了第 4.1.2 节中概述的执行策略。

重新计算允许 Kelle 在片上存储更多令牌，从而减少 DRAM 访问。处理 LLaMA2-7B 模型时，从 DRAM 访问一个 KV 向量大约需要 $1.1\mu s$ 。相比之下，使用 RSA 重新计算 KV 向量会带来 $3.2\mu s$ 的额外延迟。重新计算通过将计算与内存访问重叠来帮助隐藏内存停顿，从而减少总体延迟并将能源效率平均提高 25%。例如，从 DRAM 加载四个 KV 向量需要 $4.4\mu s$ 。通过重新计算，加载三个向量，并在加载期间并行重新计算一个向量，将总延迟减少到 $3.3\mu s$ 。在能量方面，无论输入向量的数量有多少，RSA 都保持活动状态，因此重新计算的增量能量成本可以忽略不计。

图 16 (a) 展示了三种设置下 Kelle 的屋顶线模型：No Recom (没有重新计算)、Recomp (具有中等的重新计算工作量) 和 Over Recom (具有过多的重新计算工作量)。

Table 8: Energy efficiency across refresh intervals.

LLaMA3.2-3B Interval (μ s)	Task	
	TriviaQA	PG19
1050	3.91×	8.07×
525	3.65×	7.31×
131	3.06×	6.05×

Table 9: Energy efficiency across different Batch Sizes.

Task Batch size	PG			
	Original +SRAM	AEP +SRAM	AERP +SRAM	Kelle +eDRAM
16	1×	3.16×	4.33×	6.67×
4	1×	1.71×	1.81×	2.23×
1	1×	1.24×	1.36×	1.71×

recomputation). Recomputation improves performance by increasing the effective memory bandwidth. However, as more KV vectors are recomputed, the RSA becomes a bottleneck. This behavior is reflected in the Over Recomp line, where Kelle transitions from a memory-bound regime to a compute-bound regime.

8.3.3 Impact of 2DRP and Kelle Scheduler. We evaluate Kelle+eDRAM running the Llama2-7B model executing the PG19 task under four strategies. **Org** strategy refreshes the eDRAM at its retention time with a 45μ s interval, ensuring almost no data corruption. **Uni** strategy uses a uniform refresh interval of $0.36ms$, the interval that enables the same LLM accuracy achieved by 2DRP. 2DRP, denoted as **2D** applies varying refresh intervals based on attention scores and bit positions. **2K** strategy combines both 2DRP and the Kelle scheduler. As depicted in Figure 15 (b), the finer-grained refresh policy in 2DRP improves energy efficiency. With both 2DRP and Kelle scheduler, the Kelle achieves the optimal performance.

8.3.4 Impact of eDRAM Retention Time. We assess the impact of eDRAM retention time on Kelle’s performance, considering its influence on the bit failure rate. Retention time is affected by various factors, including design, technology nodes, and temperature [29, 38, 97]. We evaluate Kelle+eDRAM on the TriviaQA and PG19 tasks using 2DRP with different retention times. Specifically, we reduce the Kelle retention time (45μ s) to average refresh intervals of 525μ s, 262μ s, and 131μ s, respectively. Table 8 shows the energy efficiency of these two settings compared to the Original+SRAM system. Thanks to AERP, the KV cache access overhead remains a small fraction of the total energy consumption. Consequently, the energy increase due to the retention time decrease is small, allowing Kelle+eDRAM to maintain performance gains.

8.3.5 Impact of Input Sequence Length. We evaluate the energy consumption of Kelle+eDRAM under long input sequence lengths using the Llama2-7B model on the PG-19 dataset across different input output sequence lengths. We use the format input length - output length (e.g., "16K-128") to denote each experiment setting. As shown in Figure 16 (b), when the input sequence is long and the decoding length is short, the prefilling stage dominates the overall energy consumption and the system becomes compute-bound. In this case, Kelle achieves a moderate energy efficiency improvement of $2.1\times$ over the Original+SRAM baseline. As both the input and output sequence lengths increase, the DRAM access energy for activations grows correspondingly. Under this more memory-intensive scenario, Kelle delivers an average energy efficiency improvement of $5.6\times$ over Original+SRAM and $1.8\times$ over AERP+SRAM, owing to its efficient KV cache management strategies.

8.3.6 Impact of Batch Size. We compare Kelle performance across different batch sizes using the Llama2-7B model on the PG-19

dataset, as shown in Table 9. While the energy efficiency improvement of Kelle over the Original+SRAM baseline is less significant at smaller batch sizes due to reduced utilization of the RSA and lower data transfer efficiency for model weights, Kelle still consistently outperforms all baselines. At a batch size of 1, Kelle achieves a speedup of 71% over Original+SRAM, 37% over AEP+SRAM, and 25% over AERP+SRAM.

8.3.7 Impact of eDRAM Bandwidth. We conduct experiments to evaluate Kelle under reduced eDRAM bandwidth ($128GB/s$), achieved by halving the number of banks and doubling the capacity per bank, while keeping the total eDRAM area and capacity constant. Using the Llama2-7B model on PG-19 and TriviaQA, Kelle+eDRAM achieves $1.47\times$ and $1.35\times$ energy gains over AERP+SRAM, and $6.31\times$ and $5.42\times$ over Original+SRAM. Though slightly lower than full-bandwidth Kelle, these results show that increasing eDRAM capacity, even with reduced bandwidth, effectively cuts costly DRAM accesses and improves bandwidth efficiency.

8.4 Discussion

8.4.1 Handle Long-Context Inference. For long context inference, due to limited eDRAM capacity, excess KV data is offloaded to 16 GB DRAM. A simple analysis with LLaMA 2 7B shows that Kelle can support up to 19,000 input tokens without AERP, assuming 8-bit weights occupy 6.5 GB out of 16 GB DRAM and each token’s 16-bit KV pair across 32 layers. Introducing AERP enables immediate KV cache reduction after each layer’s execution, freeing memory to accommodate the full input sequence in later layers. This allows Kelle to support input sequences of up to around 60K tokens. Additionally, quantizing KV vectors to 4-bit enables support for up to 240K tokens. While an upper limit remains, it exceeds typical LLM input lengths up to tens of thousands of tokens [7, 73, 75].

Although longer input sequences increase overhead, the permutation invariant property of Equations 1 and 2 allows new KV vectors to be placed in the same positions as evicted ones, greatly simplifying the paging process. Additionally, the vectors can be prefetched sequentially without requiring complex lookup mechanisms. As a result, the prefetching overhead increases linearly with the input length, avoiding disproportionate growth.

8.4.2 Integrate Kelle with GPU. While Kelle is implemented with a systolic array, AERP can be adapted to GPUs; however, identifying the token with the lowest attention score may be inefficient due to the lack of a systolic evictor. 2DRP is specific to eDRAM to reduce the refresh energy consumption. The eDRAM can be coupled with the GPU’s existing memory system to store KV vectors. Finally, the Kelle scheduler can be readily implemented using CUDA.

9 Conclusion

The KV caching technique is crucial for enhancing the efficiency of LLMs. However, storing the extensive KV vectors results in a substantial memory footprint and increased data access costs. In this work, we introduce Kelle system that utilizes eDRAM as the primary storage medium for KV vectors. The superior performance of Kelle highlights the significant potential of eDRAM in implementing the KV caching mechanism, paving the way for future research.

表 8: 刷新闻隔内的能源效率。

LLaMA3.2-3B Interval (μ s)	Task	
	TriviaQA	PG19
1050	3.91×	8.07×
525	3.65×	7.31×
131	3.06×	6.05×

表 9: 不同批量大小的能源效率。

Task	PG			
	Original +SRAM	AEP +SRAM	AERP +SRAM	Kelle +eDRAM
Batch size				
16	1×	3.16×	4.33×	6.67×
4	1×	1.71×	1.81×	2.23×
1	1×	1.24×	1.36×	1.71×

重新计算)。重新计算通过增加有效内存带宽来提高性能。然而,随着更多的 KV 向量被重新计算, RSA 成为瓶颈。这种行为反映在 Over Recomp 系列中,其中 Kelle 从内存限制状态过渡到计算限制状态。

8.3.3 2DRP 和 Kelle Scheduler 的影响。我们评估了运行 Llama2-7B 模型的 Kelle+eDRAM,该模型在四种策略下执行 PG19 任务。Org 策略以 45μ s 间隔按保留时间刷新 eDRAM,确保几乎没有数据损坏。Uni 策略使用统一的刷新闻隔 $0.36ms$,该间隔可以实现与 2DRP 相同的 LLM 精度。2DRP,表示为 2D,根据注意力分数和位位置应用不同的刷新闻隔。2K 策略结合了 2DRP 和 Kelle 调度器。如图 15 (b) 所示,2DRP 中更细粒度的刷新策略提高了能源效率。结合 2DRP 和 Kelle 调度器, Kelle 实现了最佳性能。

8.3.4 eDRAM 保留时间的影响。我们评估了 eDRAM 保留时间对 Kelle 性能的影响,考虑到它对位故障率的影响。保留时间受多种因素影响,包括设计、技术节点和温度[29,38,97]。我们使用具有不同保留时间的 2DRP 在 TriviaQA 和 PG19 任务上评估 Kelle+eDRAM。具体来说,我们将 Kelle 保留时间 (45μ s) 分别减少到平均刷新闻隔 525μ s、 262μ s 和 131μ s。表 8 显示了这两种设置与原始 SRAM 系统相比的能效。得益于 AERP, KV 缓存访问开销仍然只占总能耗的一小部分。因此,由于保留时间减少而导致的能量增加很小,从而使 Kelle+eDRAM 能够保持性能增益。

8.3.5 输入序列长度的影响。我们使用 Llama2-7B 模型在不同输入输出序列长度的 PG-19 数据集上评估长输入序列长度下 Kelle+eDRAM 的能耗。我们使用输入长度-输出长度的格式(例如“16K-128”)来表示每个实验设置。如图 16 (b) 所示,当输入序列较长且解码长度较短时,预填充阶段在总体能耗中占主导地位,系统变得计算受限。在这种情况下, Kelle 的能效比原始 SRAM 基线提高了 $2.1\times$ 。随着输入和输出序列长度的增加,用于激活的 DRAM 访问能量也相应增加。在这种内存密集型场景下,由于其高效的 KV 缓存管理策略, Kelle 的平均能效比原始 SRAM 提高了 $5.6\times$,比 AERP+SRAM 提高了 $1.8\times$ 。

8.3.6 批量大小的影响。我们使用 PG-19 上的 Llama2-7B 模型比较不同批量大小的 Kelle 性能

数据集,如表 9 所示。尽管由于 RSA 的利用率降低以及模型权重的数据传输效率较低, Kelle 相对于原始+SRAM 基线的能效改进在较小批量大小下不太显着,但 Kelle 仍然始终优于所有基线。在批量大小为 1 时, Kelle 比原始+SRAM 加速了 71%,比 AEP+SRAM 加速了 37%,比 AERP+SRAM 加速了 25%。

8.3.7 eDRAM 带宽的影响。我们进行了实验,在降低 eDRAM 带宽 (128GB/s) 的情况下评估 Kelle,通过将存储体数量减半并将每个存储体的容量加倍来实现,同时保持 eDRAM 总面积和容量不变。在 PG-19 和 TriviaQA 上使用 Llama2-7B 模型, Kelle+eDRAM 比 AERP+SRAM 实现了 $1.47\times$ 和 $1.35\times$ 能量增益,比原始+SRAM 实现了 $6.31\times$ 和 $5.42\times$ 能量增益。虽然略低于全带宽 Kelle,但这些结果表明,即使带宽减少,增加 eDRAM 容量也能有效减少昂贵的 DRAM 访问并提高带宽效率。

8.4 讨论

8.4.1 处理长上下文推理。对于长上下文推理,由于 eDRAM 容量有限,多余的 KV 数据被卸载到 16 GB DRAM。使用 LLaMA 2 7B 进行的简单分析表明, Kelle 在没有 AERP 的情况下可以支持多达 19,000 个输入令牌,假设 8 位权重占用 16 GB DRAM 中的 6.5 GB,并且每个令牌的 16 位 KV 对跨 32 层。引入 AERP 可以在每层执行后立即减少 KV 缓存,从而释放内存以容纳后面层中的完整输入序列。这使得 Kelle 能够支持最多大约 60K 个令牌的输入序列。此外,将 KV 向量量化为 4 位可以支持多达 240K 的令牌。虽然上限仍然存在,但它超出了典型的 LLM 输入长度,高达数万个标记 [7,73,75]。

虽然较长的输入序列会增加开销,但方程 1 和 2 的排列不变性允许新的 KV 向量放置在与被逐出的 KV 向量相同的位置,从而大大简化了分页过程。此外,向量可以按顺序预取,不需要复杂的查找机制。因此,预取开销随着输入长度线性增加,避免了不成比例的增长。

8.4.2 将 Kelle 与 GPU 集成。虽然 Kelle 是通过脉动阵列实现的,但 AERP 可以适应 GPU;然而,由于缺乏收缩期驱动器,识别具有最低注意力分数的令牌可能效率低下。2DRP 是 eDRAM 特有的,用于降低刷新能耗。eDRAM 可以与 GPU 现有的内存系统相结合来存储 KV 向量。最后, Kelle 调度程序可以使用 CUDA 轻松实现。

9 结论

KV 缓存技术对于提高 LLM 的效率至关重要。然而,存储大量的 KV 向量会导致大量的内存占用并增加数据访问成本。在这项工作中,我们介绍了 Kelle 系统,该系统利用 eDRAM 作为 KV 向量的主要存储介质。Kelle 的优越性能凸显了 eDRAM 在实现 KV 缓存机制方面的巨大潜力,为未来的研究铺平了道路。

References

- [1] [n. d.]. Nangate freepdk45 open cell library. <https://silvaco.com/services/library-design/>
- [2] [n. d.]. NVIDIA Jetson Orin. <https://www.siliconhighwaydirect.com/product-p/900-13767-0000-000.htm>
- [3] [n. d.]. pynvml: Python Bindings for the NVIDIA Management Library. <https://pypi.org/project/pynvml/>
- [4] Griffin Adams, Faisal Ladhak, Hailey Schoelkopf, and Raja Biswas. 2024. Cold Compress: A Toolkit for Benchmarking KV Cache Compression Approaches. <https://www.answer.ai/posts/2024-08-01-cold-compress.html>
- [5] Aditya Agrawal, Amin Ansari, and Josep Torrellas. 2014. Mosaic: Exploiting the spatial locality of process variation to reduce refresh energy in on-chip eDRAM modules. In *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 84–95.
- [6] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian I. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456* (2024).
- [7] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-v1 technical report. *arXiv preprint arXiv:2502.13923* (2025).
- [8] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee, and Vaishnav Srinivas. 2017. CACTI 7: New Tools for Interconnect Exploration in Innovative Off-Chip Memories. *ACM Trans. Archit. Code Optim.* 14, 2, Article 14 (June 2017), 25 pages. <https://doi.org/10.1145/3085572>
- [9] B. Jayant Baliga. 2019. Synopsys. *Wide Bandgap Semiconductor Power Devices* (2019). <https://api.semanticscholar.org/CorpusID:239327327>
- [10] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2019. PIQA: Reasoning about Physical Commonsense in Natural Language. In *AAAI Conference on Artificial Intelligence*. <https://api.semanticscholar.org/CorpusID:208290939>
- [11] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [12] Fenglong Cai, Dong Yuan, Zhe Yang, and Lizhen Cui. 2024. Edge-llm: A collaborative framework for large language model serving in edge computing. In *2024 IEEE International Conference on Web Services (ICWS)*. IEEE, 799–809.
- [13] Mu-Tien Chang, Paul Rosenfeld, Shih-Lien Lu, and Bruce Jacob. 2013. Technology comparison for large last-level caches (L3Cs): Low-leakage SRAM, low write-energy STT-RAM, and refresh-optimized eDRAM. In *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. 143–154. <https://doi.org/10.1109/HPCA.2013.6522314>
- [14] Jiasi Chen and Xukan Ran. 2019. Deep learning with edge computing: A review. *Proc. IEEE* 107, 8 (2019), 1655–1674.
- [15] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. DaDianNao: A Machine-Learning Supercomputer. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. 609–622. <https://doi.org/10.1109/MICRO.2014.58>
- [16] Kyungsang Cho, Yongjun Lee, Young H Oh, Gyoo-cheol Hwang, and Jae W Lee. 2014. eDRAM-based tiered-reliability memory with applications to low-power frame buffers. In *Proceedings of the 2014 international symposium on Low power electronics and design*. 333–338.
- [17] Ki Chul Chun, Pulkat Jain, Jung Hwa Lee, and Chris H. Kim. 2011. A 3T Gain Cell Embedded DRAM Utilizing Preferential Boosting for High Density and Low Power On-Die Caches. *IEEE Journal of Solid-State Circuits* 46, 6 (2011), 1495–1505. <https://doi.org/10.1109/JSSC.2011.2128150>
- [18] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *ArXiv abs/1803.05457* (2018). <https://api.semanticscholar.org/CorpusID:3922816>
- [19] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers. *arXiv preprint arXiv:2105.03011* (2021).
- [20] Jyotikrishna Dass, Shang Wu, Huihong Shi, Chaojian Li, Zhifan Ye, Zhongfeng Wang, and Yingyan Lin. 2022. ViTALiTy: Unifying Low-rank and Sparse Approximation for Vision Transformer Acceleration with a Linear Taylor Attention. *arXiv:2211.05109 [cs.CV]* <https://arxiv.org/abs/2211.05109>
- [21] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith,

Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelfer van der Linde, Jennifer Billock, Jenny Hong, Jency Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur Celebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohit Girdhar, Rohit Patil, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoiang Nie, Sharan Narang, Sharath Raparthi, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkrez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaoqing Ellen Tan, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aaron Grattafiori, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alex Vaughan, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Franco, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Firat Ozgenel, Francesco Caggioni, Francisco Guzmán, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Karthik Prasad, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelen, Keqian Li, Kun Huang, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabisa, Manav Avalani, Manish Bhatt, Maria Tsimpoukelli, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikolay Pavlovich Laptev, Ning Dong, Ning Zhang, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rit-tner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Raymond Li, Rebekkah

参考

- [1] [n. d.]. Nangate freepdk45 开放单元库. <https://silvaco.com/services/library-design/> [2] [n. d.]. NVIDIA Jetson Orin. <https://www.siliconhighwaydirect.com/product-p/900-13767-0000-000.htm>. [3] [n. d.]. pynvml: NVIDIA 管理库的 Python 绑定. <https://pypi.org/project/pynvml/>. [4] Griffin Adams, Faisal Ladhak, Hailey Schoelkopf and Raja Biswas. 2024.冷压缩: 用于对 KV 缓存压缩方法进行基准测试的工具包. <https://www.answer.ai/posts/2024-08-01-cold-compress.html> [5] Aditya Agrawal, Amin Ansari and Josep Torrellas. 2014. Mosaic: 利用工艺变化的空间局部性来减少片上 eDRAM 模块的刷新能量. 2014 年 IEEE 第 20 届高性能计算机架构国际研讨会 (HPCA). IEEE, 84–95. [6] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Martin Jaggi, Dan Ali starh, Torsten Hoeffler and James Hensman. 2024.Quarot: 旋转 llms 中的无离群值 4 位推理. arXiv 预印本 arXiv:2404.00456 (2024). [7] 白帅, 陈克勤, 刘学静, 王家林, 葛文斌, 宋思波, 党凯, 王鹏, 王世杰, 唐军, 等. 2025.Qwen 2. 5-vl 技术报告. arXiv 预印本 arXiv:2502.13923 (2025). [8] Rajeev Balasubramanian, Andrew B. Kahng, Naveen Muralimanohar, Ali Shafiee and Vaishnav Srinivas. 2017. CACTI 7: 创新片外存储器中互连探索的新工具. ACM 翻译. 建筑师. 代码优化. 14, 2, 第 14 条 (2017 年 6 月), 25 页. <https://doi.org/10.1145/3085572> [9] B. Jayant Baliga. 2019. 新思科技. 宽禁带半导体功率器件 (2019). <https://api.semanticscholar.org/CorpusID:239327327> [10] Yonatan Bisk and Rowan Zellers, Ronan Le Bras, Jianfeng Taka and Yejin Choi. 2019.PIQA: 自然语言中物理常识的推理. 在 AAAI 人工智能大会上. <https://api.semanticscholar.org/CorpusID:208290939> [11] 汤姆·B·布朗. 2020 年. 语言模型是小样本学习者. arXiv 预印本 arXiv:2005.14165 (2020). [12]蔡凤龙, 袁东, 杨哲, 崔丽珍. 2024. Edge-llm: 服务于边缘计算的大型语言模型的协作框架. 2024 年 IEEE 国际网络服务会议 (ICWS). IEEE, 799–809. [13] Mu-Tien Chang, Paul Rosenfeld, Shih-Lien Lu and Bruce Jacob. 2013 年. 大型末级缓存 (L3C) 的技术比较: 低泄漏 SRAM、低写入能耗 STT-RAM 和刷新优化 eDRAM. 2013 年 IEEE 第 19 届高性能计算机架构国际研讨会 (HPCA). 143–154. <https://doi.org/10.1109/HPCA.2013.6522314> [14]陈嘉思和冉旭侃. 2019. 深度学习与边缘计算: 回顾. 过程. IEEE 107, 8 (2019), 1655–1674. [15] 陈云吉, 罗涛, 刘少力, 张世锦, 何立强, 王佳, 李玲, 陈天石, 徐志伟, 孙宁辉, Olivier Temam. 2014.DaDianNao: 机器学习超级计算机. 2014 年第 47 届 IEEE/ACM 国际微架构研讨会. 609–622. <https://doi.org/10.1109/MICRO.2014.58> [16] Kyungsang Cho, Yongjun Lee, Young H Oh, Gyo-cheol Hwang and Jae W Lee. 2014 年. 基于 eDRAM 的分层可靠性存储器, 应用于低功耗缓冲缓冲区. 2014 年低功耗电子与设计国际研讨会论文集. 333–338. [17] Ki Chul Chun, Pulkrit Jain and Jung Hwa Lee and Chris H. Kim. 2011. 3T 增益单元嵌入式 DRAM, 利用优先升压实现高密度和低功耗片上高速缓存. IEEE 固态电路杂志 46, 6 (2011), 1495–1505. <https://doi.org/10.1109/JSSC.2011.2128150> [18] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick and Oyvind Tafjord. 2018. 你认为你已经解决了问答问题吗? 尝试 ARC, AI2 推理挑战赛. ArXiv abs/1803.05457 (2018). <https://api.semanticscholar.org/CorpusID:3922816> [19] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith and Matt Gardner. 2021. 研究论文中的信息搜索问题和答案数据集. arXiv 预印本 arXiv:2105.03011 (2021). [20] Jyotikrishna Dass, 吴尚, 史惠红, 李超建, 叶志凡, 王中峰, 林英艳. 2022.VITALiTy: 用线性泰勒注意力统一视觉变换器加速的低秩和稀疏近似. arXiv:2211.05109 [cs.CV] <https://arxiv.org/abs/2211.05109> [21] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloé Bi, Chris Marra, 克里斯·麦康奈尔, 克里斯蒂安·凯勒, 克里斯托弗·格雷, 吴春阳, 科琳·黄, 克里斯蒂安·坎顿·费尔, 赛勒斯·尼古拉迪斯, 达米安·阿隆修斯, 丹尼尔·宋, 丹妮尔·平茨, 丹尼·李

菲利普·拉德诺维奇、弗兰克·张、加布里埃尔·辛奈夫、加布里埃尔·李、乔治·刘易斯·安德森、格雷姆·内尔、格雷瓜尔·米隆、关茂、吉列姆·库雷尔、海莉·阮、汉娜·科雷瓦、胡旭、雨果·图夫龙、伊利扬·扎罗夫、伊曼诺·阿列塔·伊瓦拉、伊莎贝尔·克鲁曼、伊山·米斯拉、伊万·埃夫莫夫、杰德·科佩特、Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadekar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Janya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnston, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Lauren Rantala-Yearry, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lucas Brown, Lucas Colton, 卢卡斯·扎特, 卢克·德·奥利维拉, 马德琳·穆兹, 马赫什·帕苏普莱, 曼纳特·辛格, 马诺哈尔·帕鲁里, 马辛·卡达斯, 马修·奥尔德里奇, 马蒂厄·丽塔, 玛雅·帕夫洛娃, 梅兰妮·坎巴杜尔, 迈克·刘易斯, 敏斯, 米特什·库马尔·辛格, 莫娜·哈桑, 纳曼·戈亚尔, 纳尔吉斯·托拉比, 尼古拉·巴什利科夫, Nikolay Bogoychev, Niladri Chatterji, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, 徐朴新, 何庆, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Carl Decker, 里卡多·西尔维拉·卡布拉尔、罗伯特·斯托伊尼克、罗伯塔·莱亚努、罗希特·吉德哈、罗希特·帕特尔、罗曼·索维斯特、罗尼·波利多罗、Roshan Sumbaly、罗斯·泰勒、阮·席尔瓦、侯锐、王锐、Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, 袁少良、Sharan Narang, Sharath Rapparth, 申胜, 万盛业, Shruti Bhosale, 张顺、Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stéphane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, 童晓, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Goguet, Virginie Do, Vish Vogeti, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Weyi Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiao-qing Ellen Tan, Xinfeng Xie, Xuchao Jia, 王学伟, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, 文怡, 宋一文, 张雨辰, 李悦, 毛宇宇, Zachary Delpierre Coudert, 郑彦, 正兴陈, Zoe Papakipos, Aaditya Singh, Aaron Gattaffari, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Mehta, 阿贾伊·夏尔马, 亚历克斯·博森伯格, 亚历克斯·沃恩, 阿列克谢·巴耶夫斯基, 艾莉·范斯坦, 阿曼达·卡莱特, 阿米特·桑加尼, 阿南·尤努斯, 安德烈·卢普, 安德烈斯·阿尔瓦拉多, 安德鲁·卡普尔斯, 安德鲁·顾, 安德鲁·何, 安德鲁·波顿, 安德鲁·瑞安, 安德烈·拉姆昌达尼, 安妮·弗兰科, 阿帕拉吉塔·萨拉夫, 阿尔卡班杜·乔杜里, 阿什利·加布里埃尔, 阿什温·巴兰, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Bur-ton, Catalina Mejia, 王长瀚, 金昌奎, 周潮, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Damon Civin, Dana Beaty, Daniel Kreymier, Daniel Li, Danny Wyatt, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, 达斯坦·霍兰德, 爱德华·道林, 艾莎·贾米尔, 伊莲·蒙哥马利, 埃莱奥诺拉·普雷萨尼, 艾米丽·哈恩, 艾米丽·伍德, 埃里克·布林克曼, 埃斯特班·阿尔科特, 埃文·邓巴, 埃文·斯莫瑟斯, 孙飞, 菲利克斯·克鲁克, 田丰, 菲拉特·奥兹格内尔, 弗朗西斯科·卡吉奥尼, 弗朗西斯科·古兹曼, 弗兰克·卡纳特, 弗兰克·塞德, 加布里埃尔·梅迪纳·弗洛雷斯, 加布里埃尔·施瓦茨, Gada Badeer, Georgia Sweet, Gil Halpern, Govind Thattai, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayana, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Ibrahim Damlaj, Igor Molybog, Igor图法诺夫, 伊琳娜·埃琳娜·维利切, 伊泰·盖特, 杰克·韦斯曼, 詹姆斯·格博斯基, 詹姆斯·科利, 贾菲特·阿舍, 让-巴蒂斯特·加亚, 杰夫·马库斯, 杰夫·唐, 詹妮弗·陈, 珍妮·珍, 杰里米·雷森斯坦, 杰里米·特博尔, 杰西卡·钟, 金健, 杨静怡, 乔·卡明斯, 乔恩·卡维尔, 乔恩·谢泼德, 乔纳森·麦克菲, 乔纳森·托雷斯, 乔什·金斯伯格, 王俊杰, 吴凯, Kam Hou U, Karan Saxena, Karthik Prasad, Kartikay Khadewal, Katayoun Zand, Kathy Matosch, Kaushik Veeraraghavan, Kelly Michelena, 李克谦, 黄昆, Kunal Chawla, Kushal Lakhotia, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee贝尔, 张雷, 郭良鹏, 于立成, 利隆·莫什科维奇, 卢卡·韦尔斯特, 马迪安·哈巴萨, 马纳夫·阿瓦拉尼, 曼尼什·巴特, 玛丽亚·钦普凯利, 马丁·纳斯·曼库斯, 马

- Hogan, Robin Battey, Rocky Wang, Rohan Maheswari, Russ Howes, Rutu Rinott, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Kohler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vitor Albiero, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaofang Wang, Xiaoqian Wu, Xiaolan Wang, Xide Xia, Xilun Wu, Xinbo Gao, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yuchen Hao, Yundi Qian, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosenbriek, Zhaoduo Wen, Zhenyu Yang, and Zhiwei Zhao. 2024. The Llama 3 Herd of Models. *arXiv:2407.21783 [cs.AI]* <https://arxiv.org/abs/2407.21783>
- [22] EE Times. 2013. *Intel eDRAM Attacks Graphics in Pre 3-D IC Days*. http://www.eetimes.com/document.asp?doc_id=1263303
- [23] Chao Fang, Shouliang Guo, Wei Wu, Jun Lin, Zhongfeng Wang, Ming Kai Hsu, and Lingzhi Liu. 2022. An efficient hardware accelerator for sparse transformer neural networks. In *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2670–2674.
- [24] Eric J Fluhr, Steve Baumgartner, David Boerstler, John F Bulzacchelli, Timothy Diemond, Daniel Dreps, George English, Joshua Friedrich, Anne Gattiker, Tilman Gloekler, et al. 2014. The 12-core power8™ processor with 7.6 tb/s io bandwidth, integrated voltage regulation, and resonant clocking. *IEEE Journal of Solid-State Circuits* 50, 1 (2014), 10–23.
- [25] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [26] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2021. *A framework for few-shot language model evaluation*. <https://doi.org/10.5281/zenodo.5371628>
- [27] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs. *ArXiv abs/2310.01801* (2023). <https://api.semanticscholar.org/CorpusID:263609075>
- [28] Robert Gitterman, Amir Shalom, Andreas Burg, Alexander Fish, and Adam Teman. 2020. A 1-Mbit Fully Logic-Compatible 3T Gain-Cell Embedded DRAM in 16-nm FinFET. *IEEE Solid-State Circuits Letters* 3 (2020), 110–113. <https://doi.org/10.1109/LSSC.2020.3006496>
- [29] Robert Gitterman, Amir Shalom, Andreas Burg, Alexander Fish, and Adam Teman. 2020. A 1-Mbit Fully Logic-Compatible 3T Gain-Cell Embedded DRAM in 16-nm FinFET. *IEEE Solid-State Circuits Letters* 3 (2020), 110–113. <https://doi.org/10.1109/LSSC.2020.3006496>
- [30] Cong Guo, Jiaming Tang, Weiming Hu, Jingwen Leng, Chen Zhang, Fan Yang, Yun-Bo Liu, Minyi Guo, and Yuhao Zhu. 2023. OliVe: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization. *Proceedings of the 50th Annual International Symposium on Computer Architecture* (2023). <https://api.semanticscholar.org/CorpusID:258179335>
- [31] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).
- [32] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *arXiv:2401.18079 [cs.LG]* <https://arxiv.org/abs/2401.18079>
- [33] Yunhai Hu, Zining Liu, Zhenyuan Dong, Tianfan Peng, Bradley McDanel, and Sai Qian Zhang. 2025. Speculative decoding and beyond: An in-depth survey of techniques. *arXiv preprint arXiv:2502.19732* (2025).
- [34] Yunhai Hu, Tianhua Xia, Zining Liu, Rahul Raman, Xingyu Liu, Bo Bao, Eric Sather, Vithursan Thangarasa, and Sai Qian Zhang. 2025. DREAM: Drafting with Refined Target Features and Entropy-Adaptive Cross-Attention Fusion for Multimodal Speculative Decoding. *arXiv preprint arXiv:2505.19201* (2025).
- [35] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv:2310.06825 [cs.CL]* <https://arxiv.org/abs/2310.06825>
- [36] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551* (2017).
- [37] Kingston Technology. 2024. *Embedded Flash and DRAM Components*. <https://www.kingston.com/en/solutions/embedded-and-industrial>
- [38] W. Kong, P. C. Parries, G. Wang, and S. S. Iyer. 2008. Analysis of Retention Time Distribution of Embedded DRAM - A New Method to Characterize Across-Chip Threshold Voltage Variation. In *2008 IEEE International Test Conference*. 1–7. <https://doi.org/10.1109/TEST.2008.4700556>
- [39] HT Kung. 1986. Memory requirements for balanced computer architectures. *ACM SIGARCH Computer Architecture News* 14, 2 (1986), 49–54.
- [40] Jungi Lee, Wonbeom Lee, and Jaewoong Sim. 2024. Tender: Accelerating Large Language Models via Tensor Decomposition and Runtime Requantization. *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)* (2024), 1048–1062. <https://api.semanticscholar.org/CorpusID:270620037>
- [41] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. {InfiniGen}: Efficient generative inference of large language models with dynamic {KV} cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 155–172.
- [42] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*. PMLR, 19274–19286.
- [43] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. *arXiv:2109.07958 [cs.CL]* <https://arxiv.org/abs/2109.07958>
- [44] Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. *arXiv preprint arXiv:2405.14366* (2024).
- [45] Hanxiao Liu, Zihang Dai, David So, and Quoc V Le. 2021. Pay attention to mlps. *Advances in neural information processing systems* 34 (2021), 9204–9215.
- [46] Lian Liu, Haimeng Ren, Long Cheng, Zhaohui Xu, Yudong Pan, Mengdi Wang, Xiaowei Li, Yinhe Han, and Ying Wang. 2024. COMET: Towards Partial W4A4KV4 LLMs Serving. *arXiv:2410.12168 [cs.AR]* <https://arxiv.org/abs/2410.12168>
- [47] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyriillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the Persistence of Importance Hypothesis for LLM KV Cache Compression at Test Time. *ArXiv abs/2305.17118* (2023). <https://api.semanticscholar.org/CorpusID:258947558>
- [48] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ArXiv abs/2402.02750* (2024). <https://api.semanticscholar.org/CorpusID:267413049>
- [49] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen (Henry) Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: a tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML '24)*. JMLR.org, Article 1311, 13 pages.
- [50] Siyuan Lu, Meiqi Wang, Shuang Liang, Jun Lin, and Zhongfeng Wang. 2020. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer. In *2020 IEEE 33rd International System-on-Chip Conference (SOCC)*. IEEE, 84–89.
- [51] Bradley McDanel, Sai Qian Zhang, Yunhai Hu, and Zining Liu. 2025. PipeSpec: Breaking Stage Dependencies in Hierarchical LLM Decoding. *arXiv preprint arXiv:2505.01572* (2025).
- [52] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. *arXiv:1609.07843 [cs.CL]*
- [53] Ramesh Nallapati, Bowen Zhou, Cicero Nogueira dos santos, Caglar Gulcehre, and Bing Xiang. 2016. Abstractive Text Summarization Using Sequence-to-Sequence RNNs and Beyond. *arXiv:1602.06023 [cs.CL]* <https://arxiv.org/abs/1602.06023>
- [54] Duy-Thanh Nguyen, Nhut-Minh Ho, and Ik-Joon Chang. 2019. St-DRC: Stretchable DRAM refresh controller with no parity-overhead error correction scheme for energy-efficient DNNs. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–6.
- [55] NVIDIA Corporation. 2024. NVIDIA System Management Interface (nvidia-smi). <https://developer.nvidia.com/system-management-interface>.
- [56] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2024. Instinifer: In-storage attention offloading for cost-effective long-context llm inference. *arXiv preprint arXiv:2409.04992* (2024).
- [57] Eunhyeok Park, Dongyoung Kim, and Sungjoo Yoo. 2018. Energy-efficient neural network accelerator based on outlier-aware low-precision computation. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 688–698.
- [58] Alicia Parrish, Angelica Chen, Nikita Nangia, Vishakh Padmakumar, Jason Phang, Jana Thompson, Phu Mon Htut, and Samuel R. Bowman. 2022. BBQ: A Hand-Built Bias Benchmark for Question Answering. *arXiv:2110.08193 [cs.CL]* <https://arxiv.org/abs/2110.08193>

- 霍根、罗宾·巴蒂、洛奇·王、罗汉·马赫斯瓦里、拉斯·豪斯、鲁蒂·里诺特、赛·贾耶什·邦杜、萨米亚克·达塔、萨拉·丘格、萨拉·亨特、萨尔贡·迪伦、萨沙·西多罗夫、萨塔德鲁·潘、索拉布·维尔玛、山本精二、沙拉德·拉马斯瓦米、肖恩·林赛、肖恩·林赛、盛峰、林盛豪、查胜欣、Shiva Shankar、张书强、张书强、Sinong Wang、Sneha Agarwal、Soji Sajuyigbe、Soumith Chintala、Stephanie Max、Stephen Chen、Steve Kehoe、Steve Satterfield、Sudarshan Govindaprasad、Sumit Gupta、Sungmin Cho、Sunny Virk、Suraj Subramanian、Sy Choudhury、Sydney Goldman、Tal Remez、Tamar Glaser、Tamara Best、Thilo Kohler、Thomas Robinson、李天河、张天军、Tim Matthews、Timothy Chou、Tzook Shaked、Varun Vontimitta、Victoria Ajayi、Victoria Montanez、Vijai Mohan、Vinay Satish Kumar、Vishal Mangla、Vitor Albiero、Vlad Ionescu、Vlad Poenaru、Vlad Tiberiu Mihailescu、Vladimir Ivanov、Wei Li、Wenchen王、文文江、韦斯·布阿齐兹、威尔·康斯特布尔、汤晓成、王晓芳、吴晓建、王晓兰、夏希德、吴希伦、高新波、陈彦军、胡晔、叶佳、叶琪、李彦达、张一琳、张英、Yossi Adi、Youngjin Nam、Yu、王、郝雨辰、钱云迪、何雨子、Zach Rait、Zachary DeVito、Zef Rosnbrick、温兆铎、杨振宇、赵志伟。2024. Llama 3 模型群。arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783> [22] EE 时报。2013 年。英特尔 eDRAM 在前 3-D IC 时代攻击显卡。 http://www.eetimes.com/document.asp?doc_id=1263303 [23] 方超、郭守良、吴伟、林俊、王中峰、徐明凯、刘令智。2022. 稀疏变压器神经网络的高效硬件加速器。2022 年 IEEE 国际电路与系统研讨会 (ISCAS)。IEEE, 2670–2674. [24] Eric J Fluhr、Steve Baumgartner、David Boerstler、John F Bulzacchelli、Timothy Diemoz、Daniel Dreps、George English、Joshua Friedrich、Anne Gattiker、Tilman Gloekler 等。2014 年。12 核 power8™ 处理器，具有 7.6 tb/s io 带宽、集成电压调节和谐振时钟。IEEE 固态电路杂志 50, 1 (2014), 10–23. [25] Elias Frantar、Saleh Ashkboos、Torsten Hoeftler 和 Dan Alistarh。2022. Gptq: 生成式预训练 Transformer 的准确训练后量化。arXiv 预印本 arXiv:2210.17323 (2022). [26] Leo Gau、Jonathan Tow、Stella Biderman、Sid Black、Anthony DiPofi、Charles Foster、Laurence Golding、Jeffrey Hsu、Kyle McDonell、Niklas Muennighoff、Jason Phang、Laria Reynolds、Eric Tang、Anish Thite、Ben Wang、Kevin Wang 和 Andy Zou。2021. 小样本语言模型评估框架。 <https://doi.org/10.5281/zenodo.5371628> [27] 葛苏宇、张雨楠、刘丽媛、张敏佳、韩家伟、高剑峰。2023 年。模型告诉您放弃什么：LLM 的自适应 KV 缓存压缩。ArXiv abs/2310.01801 (2023). <https://api.semanticscholar.org/CorpusID:263609075> [28] Robert Gitterman、Amir Shalom、Andreas Burg、Alexander Fish 和 Adam Teman。2020 年。采用 16 nm FinFET 的 1 Mbit 完全逻辑兼容 3T 增益单元嵌入式 DRAM。IEEE 固态电路快报 3 (2020), 110–113. <https://doi.org/10.1109/LSSC.2020.3006496> [29] 罗伯特·吉特曼、阿米尔·沙洛姆、安德烈亚斯·伯格、亚历山大·菲什和亚当·特曼。2020 年。采用 16 nm FinFET 的 1 Mbit 完全逻辑兼容 3T 增益单元嵌入式 DRAM。IEEE 固态电路快报 3 (2020), 110–113. <https://doi.org/10.1109/LSSC.2020.3006496> [30] 郭从、唐天明、胡伟明、冷静文、张晨、杨帆、刘云波、郭敏仪、朱宇豪。2023. Olive: 通过硬件友好的异常值-受害者对量化加速大型语言模型。第 50 届计算机架构国际研讨会论文集 (2023 年)。 <https://api.semanticscholar.org/CorpusID:258179335> [31] Dan Hendrycks 和 Kevin Gimpel。2016. 高斯误差线性单位 (gelu)。arXiv 预印本 arXiv:1606.08415 (2016). [32] Coleman Hooper、Sehoon Kim、Hiva Mohammadzadeh、Michael W. Mahoney、Yakun Sophia Shao、Kurt Keutzer 和 Amir Gholami。2024. KVQuant: 通过 KV 缓存量化实现 1000 万上下文长度 LLM 推理。arXiv:2401.18079 [cs.LG] <https://arxiv.org/abs/2401.18079> [33] 胡云海、刘子宁、董振元、彭天帆、布拉德利·麦克丹尼尔和张赛千。2025. 推测解码及其他：对技术的深入调查。arXiv 预印本 arXiv:2502.19732 (2025). [34] 胡云海、夏天华、刘子宁、Rahul Raman、刘星宇、鲍波、Eric Sather、Vithursan Thangarasa、张赛千。2025. 梦想：利用细化的目标特征和熵自适应交叉注意力融合进行多模态推测解码的起草。arXiv 预印本 arXiv:2505.19201 (2025). [35] Albert Q. Jiang、Alexandre Sablayrolles、Arthur Mensch、Chris Bamford、Devendra Singh Chaplot、Diego de las Casas、Florian Bressand、Gianna Lengyel、Guillaume Lample、Lucile Saulnier、Lélio Renard Lavaud、Marie-Anne Lachaux、Pierre Stock、Teven Le Scao、Thibaut Lavril、Thomas Wang、Timothée
- [37] 金士顿科技。2024. 嵌入式闪存和 DRAM 组件。 <https://www.kingston.com/en/solutions/embedded-and-industrial> [38] W. Kong、P. C. Paries、G. Wang 和 S. S. Iyer。2008. 嵌入式 DRAM 保留时间分布分析 - 表征跨芯片阈值电压变化的新方法。2008 年 IEEE 国际测试大会。1–7. <https://doi.org/10.1109/TEST.2008.4700556> [39] HT Kung。1986. 平衡计算机体系结构的内存要求。ACM SIGARCH 计算机架构新闻 14, 2 (1986), 49–54. [40] Jungi Lee、Wonbeom Lee 和 Jaewoong Sim。2024. Tender: 通过张量分解和运行时重新量化加速大型语言模型。2024 年 ACM/IEEE 第 51 届计算机架构国际研讨会 (ISCA) (2024), 1048–1062. <https://api.semanticscholar.org/CorpusID:270620037> [41] Wonbeom Lee、Jungi Lee、Junghwan Seo 和 Jaewoong Sim。2024. {InfiniGen}: 通过动态 {KV} 缓存管理对大型语言模型进行高效生成推理。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24)。155–172. [42] Yaniv Leviathan、Matan Kalman 和 Yosi Matias。2023. 通过推测解码从 Transformer 进行快速推理。在国际机器学习会议上。PMLR, 19274–19286. [43] 斯蒂芬妮·林、雅各布·希尔顿和欧文·埃文斯。2022. TruthfulQA: 衡量模型如何模仿人类的谎言。arXiv:2109.07958 [cs.CL] <https://arxiv.org/abs/2109.07958> [44] Akide Liu、Jing Liu、Zizheng Pan、Yefei He、Gholamreza Haffari、Bohan Zhuang。2024. MiniCache: 大型语言模型深度维度的 KV 缓存压缩。arXiv 预印本 arXiv:2405.14366 (2024). [45] 刘汉晓、戴子航、苏大卫、Quoc V Le。2021 年。关注 MLPs。神经信息处理系统的进展 34 (2021), 9204–9215. [46] 刘连、任海萌、程龙、徐朝晖、潘玉东、王梦迪、李小平、韩银河、王英。2024. COMET: 面向部分 W4A4KV4 法学硕士服务。arXiv:2410.12168 [cs.AR] <https://arxiv.org/abs/2410.12168> [47] Zichang Liu、Aditya Desai、Fangshuo Liao、Weitao Wang、Victor Xie、Zhaozhao Xu、Anastasios Kyrillidis 和 Anshumali Shrivastava。2023. 剪刀手：在测试时利用 LLM KV 缓存压缩的重要性假设的持久性。ArXiv abs/2305.17118 (2023). <https://api.semanticscholar.org/CorpusID:258947558> [48] Zirui Liu、Jiayi Yuan、Hongye Jin、Shaohenzhong、Chaozhao Xu、Vladimir Braverman、Beidi Chen 和 Xia Hu。2024. KIVI: KV 缓存的免调整非对称 2 位量化。ArXiv 绝对/2402.02750 (2024). <https://api.semanticscholar.org/CorpusID:267413049> [49] Zirui Liu、Jiayi Yuan、Hongye Jin、Shaohenzhong (Henry) Zhu、Zhaozhao Xu、Vladimir Braverman、Beidi Chen 和 Xia Hu。2024. KIVI: KV 缓存的免调整非对称 2 位量化。第 41 届国际机器学习会议 (奥地利维也纳) 会议记录 (ICML' 24)。JMLR.org, 第 13 条, 13 页。[50] 陆思源、王美琪、梁爽、林俊、王中峰。2020. 用于变压器中多头注意力和位置前馈的硬件加速器。2020 年 IEEE 第 33 届国际系统会议 (SOCC)。IEEE, 84–89. [51] 布拉德利·麦克丹尼尔、张赛前、胡云海、刘子宁。2025. PipeSpec: 打破分层 LLM 解码中的阶段依赖性。arXiv 预印本 arXiv:2505.01572 (2025). [52] 斯蒂芬·梅里蒂、蔡明熊、詹姆斯·布拉德伯里和理查德·索彻。2016. 指针哨兵混合模型。arXiv:1609.07843 [cs.CL] [53] Ramesh Nallapati、Bowen Zhou、Cicero Nogueira dos santos、Caglar Gulcehre 和 Bing Xian。2016. 使用序列到序列 RNN 及其他技术进行抽象文本摘要。arXiv:1602.06023 [cs.CL] <https://arxiv.org/abs/1602.06023> [54] Duy-Thanh Nguyen、Nhu T-Minh Ho 和 Ik-Joon Chang。2019. St-DRC: 可扩展 DRAM 刷新控制器，无奇偶校验开销纠错方案，适用于节能 DNN。2019 年第 56 届年度设计自动化会议论文集。1–6. [55] 英伟达公司。2024. NVIDIA 系统管理接口 (nvidia-smi)。 <https://developer.nvidia.com/system-management-interface>. [56] 潘秀琴、李恩典、李巧、梁胜文、单一洲、周克、罗英伟、王晓琳、张杰。2024. Instinfer: 存储内注意力卸载，实现经济高效的长上下文 llm 推理。arXiv 预印本 arXiv:2409.04992 (2024). [57] Eunhyeok Park、Dongyoung Kim、Sungjoo Yoo。2018. 基于异常值感知低精度计算的节能神经网络加速器。2018 年 ACM/IEEE 第 45 届计算机体系结构国际研讨会 (ISCA)。IEEE, 688–698. [58] Alicia Parrish、Angelica Chen、Nikita Nangia、Vishakh Padmakumar、Jason Phang、Jana Thompson、Phu Mon Htut 和 Samuel R. Bowman。2022. BBQ: 手工构建的问答偏差基准。arXiv:2110.08193 [cs.CL] <https://arxiv.org/abs/2110.08193>

- [59] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems* 5 (2023), 606–624.
- [60] Matt Poremba, Sparsh Mittal, Dong Li, Jeffrey S. Vetter, and Yuan Xie. 2015. DESTINY: A tool for modeling emerging 3D NVM and eDRAM caches. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 1543–1546. <https://doi.org/10.7873/DATE.2015.0733>
- [61] Team PyTorch. 2023. Accelerating generative ai with pytorch ii: Gpt, fast. <https://pytorch.org/blog/accelerating-generative-ai-2/>
- [62] Jiajun Qin, Tianhua Xia, Cheng Tan, Jeff Zhang, and Sai Qian Zhang. 2025. PICACHU: Plug-In CGRA Handling Upcoming Nonlinear Operations in LLMs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 845–861.
- [63] Alec Radford. 2018. Improving language understanding by generative pre-training. (2018).
- [64] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models are Unsupervised Multitask Learners. <https://api.semanticscholar.org/CorpusID:160025533>
- [65] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [66] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. 2019. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507* (2019).
- [67] Haihao Shen, Hanwen Chang, Bo Dong, Yu Luo, and Hengyu Meng. 2023. Efficient LLM Inference on CPUs. *arXiv:2311.00502 [cs.LG]* <https://arxiv.org/abs/2311.00502>
- [68] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [69] Weisong Shi, Jie Cao, Quan Zhang, Youhui Li, and Lanyu Xu. 2016. Edge computing: Vision and challenges. *IEEE internet of things journal* 3, 5 (2016), 637–646.
- [70] J. Soriaga. 2023. *Accelerating Generative AI at the Edge*. <https://www.qualcomm.com/news/onq/2023/11/accelerating-generative-ai-at-the-edge> [Online; accessed 29-Mar-2025].
- [71] Jacob R. Stevens, Rangharajan Venkatesan, Steve Dai, Bruce Khailany, and Anand Raghunathan. 2021. Softmax: Hardware/Software Co-Design of an Efficient Softmax for Transformers. *arXiv:2103.09301 [cs.AR]* <https://arxiv.org/abs/2103.09301>
- [72] Manu Suryavansh. 2020. Google Coral Edge TPU Board Vs NVIDIA Jetson Nano Dev board Hardware Comparison.
- [73] Qwen Team. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* (2024).
- [74] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [75] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shriti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [76] Fengbin Tu, Weiwei Wu, Shouyi Yin, Leibo Liu, and Shaojun Wei. 2018. RANA: Towards Efficient Neural Acceleration with Refresh-Optimized Embedded DRAM. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. 340–352. <https://doi.org/10.1109/ISCA.2018.00037>
- [77] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. BitNet: Scaling 1-bit Transformers for Large Language Models. *arXiv:2310.11453 [cs.CL]* <https://arxiv.org/abs/2310.11453>
- [78] Hanrui Wang, Zhekai Zhang, and Song Han. 2020. SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning. *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2020), 97–110. <https://api.semanticscholar.org/CorpusID:229298088>
- [79] Xiaotian Wang, Teng Tian, Letian Zhao, Wei Wu, and Xi Jin. 2022. Exploration of balanced design in resource-constrained edge device for efficient CNNs. *IEEE Transactions on Circuits and Systems II: Express Briefs* 69, 11 (2022), 4573–4577.
- [80] Dieter F Wendel, Ron Kalla, James Warnock, Robert Cargnoni, Sam G Chu, Joachim G Clabes, Daniel Dreps, David Hrusecky, Josh Friedrich, Saiful Islam, et al. 2010. POWER7™, a highly parallel, scalable multi-core high end server processor. *IEEE Journal of Solid-State Circuits* 46, 1 (2010), 145–161.
- [81] Tianhua Xia and Sai Qian Zhang. 2024. Hyft: A Reconfigurable Softmax Accelerator with Hybrid Numeric Format for both Training and Inference. In *Proceedings of the 29th ACM/IEEE International Symposium on Low Power Electronics and Design (Newport Beach, CA, USA) (ISLPED '24)*. Association for Computing Machinery, New York, NY, USA, 1–6. <https://doi.org/10.1145/3665314.3670816>
- [82] Jingyang Xiang and Sai Qian Zhang. 2024. DFRot: Achieving Outlier-Free and Massive Activation-Free for Rotated LLMs with Refined Rotation. *arXiv preprint arXiv:2412.00648* (2024).
- [83] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. *arXiv:2309.17453 [cs.CL]* <https://arxiv.org/abs/2309.17453>
- [84] Kai Xiao, Jing Wan, Hui Xie, Yuxuan Zhu, Tian Tian, Wei Zhang, Yingxin Chen, Jinshu Zhang, Lihui Zhou, Sheng Dai, et al. 2024. High performance Si-MoS₂ heterogeneous embedded DRAM. *Nature Communications* 15, 1 (2024), 9782.
- [85] Xiao Xiong, Zhaorui Chen, Yue Liang, Minghao Tian, Jiaxing Shang, Jiang Zhong, and Dajiang Liu. 2025. DynaX: Sparse Attention Acceleration with Dynamic X:M Fine-Grained Structured Pruning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 260–274. <https://doi.org/10.1145/3676641.3715991>
- [86] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2024. Fast On-device LLM Inference with NPUs. *arXiv:2407.05858 [cs.AI]* <https://arxiv.org/abs/2407.05858>
- [87] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Yang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Xezhi Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuhong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. *arXiv:2407.10671 [cs.CL]* <https://arxiv.org/abs/2407.10671>
- [88] Dongjie Yang, XiaoDong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. *arXiv preprint arXiv:2405.12532* (2024).
- [89] Chengshuo Yu, Taegeun Yoo, Hyunjoon Kim, Tony Tae-Hyoung Kim, Kevin Chai Tshun Chuan, and Bongjin Kim. 2020. A logic-compatible eDRAM compute-in-memory with embedded ADCs for processing neural networks. *IEEE Transactions on Circuits and Systems I: Regular Papers* 68, 2 (2020), 667–679.
- [90] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, et al. 2024. Cambricon-llm: A chiplet-based hybrid architecture for on-device inference of 70b llm. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1474–1488.
- [91] Zhongzhi Yu, Zheng Wang, Yuhua Li, Ruijie Gao, Xiaoya Zhou, Sreenidhi Reddy Bommu, Yang Zhao, and Yingyan Lin. 2024. Edge-llm: Enabling efficient large language model adaptation on edge devices via unified compression and adaptive layer voting. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [92] Ali Hadi Zadeh, Isak Edo, Omar Mohamed Awad, and Andreas Moshovos. 2020. Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 811–824.
- [93] Hengrui Zhang, August Ning, Rohan Baskar Prabhakar, and David Wentzlaff. 2024. Llmcompass: Enabling efficient hardware design for large language model inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1080–1096.
- [94] Mingjin Zhang, Xiaoming Shen, Jiannong Cao, Zeyang Cui, and Shan Jiang. 2024. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. *IEEE Internet of Things Journal* (2024), 1–1. <https://doi.org/10.1109/JIOT.2024.3524255>
- [95] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. *arXiv:2205.01068 [cs.CL]* <https://arxiv.org/abs/2205.01068>
- [96] Sai Qian Zhang, Bradley McDanel, and H. T. Kung. 2021. FAST: DNN Training Under Variable Precision Block Floating Point with Stochastic Rounding. *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (2021), 846–860. <https://api.semanticscholar.org/CorpusID:240288620>
- [97] Sai Qian Zhang, Thierry Tambe, Nestor Cuevas, Gu-Yeon Wei, and David Brooks. 2023. CAMEL: Co-Designing AI Models and Embedded DRAMs for Efficient On-Device Learning. *arXiv:2305.03148 [cs.AR]* <https://arxiv.org/abs/2305.03148>
- [98] Zhenyu (Allen) Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark W. Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *ArXiv abs/2306.14048* (2023). <https://api.semanticscholar.org/CorpusID:259263947>

- [52] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Shaw, Shivani Agrawal and Jeff Dean. 2023.有效扩展变压器推理。机器学习系统和论文集 5 (2023), 606–624. [60] 马特·波伦巴, 斯帕什·米塔尔, 李东, 杰弗里·S·维特, 谢元. 2015. DESTINY: 用于对新兴 3D NVM 和 eDRAM 缓存进行建模的工具. 2015 年欧洲设计、自动化和测试会议暨展览 (日期). 1543–1546. <https://doi.org/10.7873/DATE.2015.0733> [61] PyTorch 团队. 2023. 使用 pytorch ii 加速生成式人工智能: Gpt, 快. <https://pytorch.org/blog/accelerating-generative-ai-2/> [62] 秦家军, 夏天华, 谭成, 张杰夫和张赛谦. 2025.PICACHU: 插件 CGRA 处理大语言模型即将到来的非线性操作. 第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 2 卷. 845–861. [63] 亚历克·雷德福. 2018.通过生成预训练提高语言理解. (2 018). [64] 亚历克·雷德福、杰夫·吴、Rewon Child、大卫·卢安、达里奥·阿莫代和伊利亚·苏茨克弗. 2019. 语言模型是无监督的多任务学习者. <https://arxiv.org/abs/160025533> [65] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever 等人. 2019. 语言模型是无监督的多任务学习者. OpenAI 博客 1, 8 (2019), 9. [66] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar and Timothy P Lillicrap. 2019. 用于远程序列建模的压缩变压器. arXiv 预印本 arXiv:1911.05507 (2019). [67] 沉海浩, 常汉文, 董波, 罗宇, 孟恒宇. 2023. CPU 上的高效 LLM 推理. arXiv:2311.00502 [cs.LG]. <https://arxiv.org/abs/2311.00502> [68] Ying Shen, Lianmin Cheng, Binhuan g Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica 和 Ce Zhang. 2023. Flexgen: 使用单个 GPU 进行大型语言模型的高吞吐量生成推理. 在国际机器学习会议上. PMLR, 31094–31116. [69] 施伟松, 曹杰, 张全, 李友慧, 徐兰宇. 2016. 边缘计算: 愿景与挑战. IEE E 物联网期刊 3, 5 (2016), 637–646. [70] J.索里亚加. 2023 年. 加速边缘生成人工智能. <https://www.qualcomm.com/news/onq/2023/11/accelerating-generative-ai-at-the-edge> [在线; 2023 年 3 月 29 日访问]. [71] Jacob R. Stevens, Ranghar ajan Venkatesan, Steve Dai, Bruce Khailany and Anand Raghunathan. 2021.Soft emax: 变压器高效 Softmax 的硬件/软件协同设计. arXiv:2103.09301 [cs.AR] <https://arxiv.org/abs/2103.09301> [72] 马努·苏里亚万什. 2020. Google Coral Edg e TPU 板与 NVIDIA Jetson Nano 开发板硬件比较. [73]Qwen团队. 2024.Qwen2 技术报告. arXiv 预印本 arXiv:2407.10671 (2024). [74] Hugo Touvron, Thibau t Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacr oix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar 等. 2023. Lla ma: 开放高效的基础语言模型. arXiv 预印本 arXiv:2302.13971 (2023). [75] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Ya smine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shrusti Bh osale 等. 2023. Llama 2: 开放基础和微调的聊天模型. arXiv 预印本 arXiv:23 07.09288 (2023). [76] 涂凤斌, 吴伟伟, 守守义, 刘雷波, 魏少军. 2018.RAN A: 利用刷新优化的嵌入式 DRAM 实现高效神经加速. 2018年ACM/IEEE第45届计算机体系结构国际研讨会 (ISCA). 340–352. <https://doi.org/10.1109/ISCA.2018.00037> [77] 王宏宇, 马树明, 董立, 黄少涵, 王怀杰, 马凌晓, 杨范 , 王瑞平, 吴一, 魏福如. 2023. BitNet: 为大型语言模型扩展 1 位 Transfome r. arXiv:2310.11453 [cs.CL] <https://arxiv.org/abs/2310.11453> [78] 王涵瑞、张哲凯和韩松. 2020.SpAtten: 具有级联令牌和头部修剪的高效稀疏注意力架构. 2021 年 IEEE 国际高性能计算机架构研讨会 (HPCA) (2020), 97–110. <https://arxiv.org/abs/2209.08088> [79] 王晓天, 田腾, 赵乐天, 吴伟, 金宇. 2022. 探索资源受限边缘设备中高效 CNN 的平衡设计. IEEE 电路与系统汇刊 II: Express Briefs 69, 11 (2022), 4573–4577. [80] Dieter F Wendel, Ron Kalla, James Warnock, Robert Cargnoni, Sam G Chu, Joachim G Clabes, Dani el Dreps, David Hrusecky, Josh Friedrich, Saiful Islam 等. 2010. POWER7™, 高度并行、可扩展的多核高端服务器处理器. IEEE 固态电路杂志 46, 1 (2010), 145–161. [81] 夏天华, 张赛前. 2024. Hyft: 一种可重新配置的 Softmax 加速器, 具有用于训练和推理的混合数字格式. 第 29 届 ACM/IEEE L 国际研讨会论文集 [82] 向景阳, 张赛前. 2024. DFRot: 通过精细旋转实现旋转大语言模型超值和大规模激活. arXiv 预印本 arXiv:2412.00648 (2024). [83] 肖光轩, 田远东, 陈蓓迪, 韩松, 麦克·刘易斯. 2024.具有注意力池的高效流式语言模型. a rXiv:2309.17453 [cs.CL] <https://arxiv.org/abs/2309.17453> [84] 肖凯, 万静, 谢慧 , 朱雨轩, 田甜, 张伟, 陈迎新, 张金树, 周丽辉, 戴胜等. 2024年.高性能 Si-MoS2 异构嵌入式 DRAM. 自然通讯 15, 1 (2024), 9782. [85] 熊晓, 陈兆瑞, 梁跃, 田明浩, 商嘉兴, 钟江, 刘大江. 2025. DynaX: 利用动态 X:M 细粒度结构化剪裁的稀疏注意力加速. 第 30 届 ACM 国际编程语言和操作系统架构支持会议论文集, 第 2 卷 (荷兰鹿特丹) (ASPLOS ’ 25). 计算机协会, 美国纽约纽约市, 260–274. <https://doi.org/10.1145/3676641.3715991> [86] 徐大亮, 张浩, 杨黎明, 刘瑞琪, 黄刚, 徐孟伟, 刘轩哲. 2024. 使用 NPU 进行快速设备上 LLM 推理. arXiv:2407.05858 [cs.AI] <https://arxiv.org/abs/2407.05858> [87] An Yang, Baosong Yang, Binyuan Hui, Bo Cheng, Bowen Yu, Chang Zho u, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, J ianwei Zhang, 马建新、杨建新、徐进、周静仁、白金泽、何金正、林俊阳、党凯、陆克明、陈克勤、杨可欣、李梅、薛明峰、倪娜、张培、王鹏、彭如、门锐、高瑞泽、林润吉、王士杰、白帅、谭思南、朱天行、李天浩、刘天宇、葛文斌、邓晓东、周小欢、任兴章、张新宇、崔希品、任宣成、刘学静、范阳 、姚洋、张宜昌、万宇、褚云飞、刘玉琼、魏泽宇、张振如、郭志芳和范志浩 . 2024.Qwen2 技术报告. arXiv:2407.10671 [cs.CL] <https://arxiv.org/abs/2407.10671> [88] 杨东杰、韩晓东、高岩、胡耀、张世林和赵海. 2024. PyramidInfer: 用于高吞吐量 LLM 推理的金字塔 KV 缓存压缩. arXiv 预印本 arXiv:2405.125 32 (2024). [89] Chengshuo Yu, Taegun Yoo, Hyunjoon Kim, Tony Tae-Hyou ng Kim, Kevin Chai Tshun Chuan and Bongjin Kim. 2020 年. 一种逻辑兼容的 e DRAM 内存计算, 具有用于处理神经网络的嵌入式 ADC. IEEE 电路与系统 汇刊 I: 常规论文 68, 2 (2020), 667–679. [90] 于仲凯, 梁胜文, 马天云, 蔡云 科, 南子源, 黄迪, 宋新凯, 郝一凡, 张杰, 田智, 等. 2024. Cambricon-llm : 一种基于小芯片的混合架构, 用于 70b llm 的设备上推理. 2024年第57届IE EE/ACM国际微架构研讨会 (MICRO). IEEE, 1474–1488. [91] 于中植, 王 政, 李雨涵, 高瑞杰, 周小亚, Sreenidhi Reddy Bommu, 赵阳, 林英桢. 202 4. Edge-llm: 通过统一压缩和自适应层投放在边缘设备上实现高效的大型语言 模型自适应. 第 61 届 ACM/IEEE 设计自动化会议论文集. 1–6. [92] 阿里·哈 迪·扎德、伊萨克·埃多、奥马尔·穆罕默德·阿瓦德和安德烈亚斯·莫索沃斯. 202 0.Gobo: 量化基于注意力的 NLP 模型, 以实现低延迟和节能推理. 2020 年第 53 届 IEEE/ACM 国际微架构研讨会 (MICRO). IEEE, 811–824. [93] 张恒瑞 , 宁奥古斯特, 罗汉·巴斯卡·普拉巴卡, 大卫·温茨拉夫. 2024. Llmcompass: 为大型语言模型推理实现高效的硬件设计. 2024 年 ACM/IEEE 第 51 届计算机 体系结构国际研讨会 (ISCA). IEEE, 1080–1096. [94] 张明金, 沉晓明, 曹 建农, 崔泽阳, 姜山. 2024. EdgeShard: 通过协作边缘计算进行高效的 LLM 推理. IEEE 物联网杂志 (2024), 1–1. <https://doi.org/10.1109/JIOT.2024.3524255> [95] Susan 张、Stephen Roller、Naman Goyal、Mikel Artetxe、Moya Chen、Sh uohui Chen、Christopher Dewan、Mona Diab、Xian Li、Xi Victoria Lin、Todor Mihaylov、Myle Ott、Sam Shleifer、Kurt Shuster、Daniel Simig、Punit Singh K oura、Anjali斯里达尔、王天禄和卢克·泽特莫耶. 2022. OPT: 开放预训练的 Transformer 语言模型. arXiv:2205.01068 [cs.CL] <https://arxiv.org/abs/2205.01068> [96] 张赛千、布拉德利·麦克丹尼尔和 H. T. Kung. 2021.FAST: 在具有随机 舍入的可变精度块浮点下进行 DNN 训练. 2022 年 IEEE 国际高性能计算机架 构研讨会 (HPCA) (2021), 846–860. <https://arxiv.org/abs/2205.01068> [97] 张赛谦、Thierry Tambe、Nestor Cuevas、Gu-Yeon Wei 和 David B rooks. 2023. CAMEL: 共同设计人工智能模型和嵌入式 DRAM, 以实现高效的 设备上学习. arXiv:2305.03148 [cs.AR] [https://arxiv.org/abs/2305.031](https://arxiv.org/abs/2305.03148)

- [99] Youpeng Zhao, Di Wu, and Jun Wang. 2024. ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching. *arXiv preprint arXiv:2403.17312* (2024).
- [100] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. Transpim: A memory-based acceleration via software-hardware co-design for transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1071–1085.

[99] 赵友鹏, 吴迪, 王军。2024. ALISA: 通过稀疏感知 KV 缓存加速大型语言模型推理。arXiv 预印本 arXiv:2403.17312 (2024)。

[100] 周敏轩, 徐伟红, Jaeyoung Kang, Tajana Rosing。2022. Transpim: 通过软件-硬件协同设计为变压器提供基于内存的加速。2022 年 IEEE 国际高性能计算机架构研讨会 (HPCA)。IEEE, 1071–1085。